

COM7021 | Lesson 1 | Data Visualisation

Data Visualisation (COM7021)

Lesson 1: Introduction to Data Visualisation

Image by FAMILY STOCK from Shutterstock.com

- Module overview
- Lesson introduction
- Topic 1: Introduction
- Topic 2: Common visualisation mistakes
- Topic 3: Basic visualisations using Matplotlib
- Topic 4: Introduction to Seaborn
- Reading List

Module overview

Welcome from your Module Team

Please watch the video below, or access the transcript, which will provide you with an overview of this Module's content and objectives.

Estimated video duration: 5.5 minutes

05:41

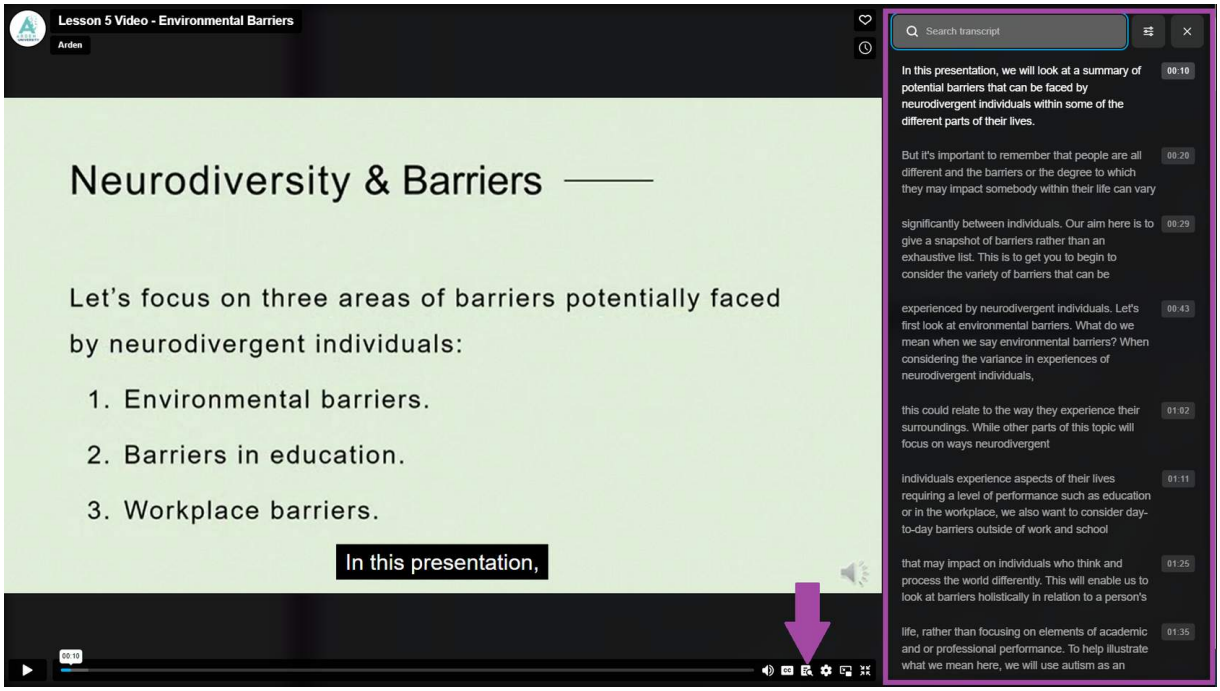
Data visualisation - Module overview

Module overview video: captions and transcript

To access the auto-generated captions, click on the 'CC/Subtitles' icon in the video player at the bottom right-hand corner.

To access the transcript, click on the third icon along from the left and this will bring up a transcript on the right-hand side, next to the video for you to view.

Image Description: Screenshot below shows an example of a Vimeo video. There is an arrow pointing to the transcript option which is the third icon in on the right-hand side underneath the video.



PDF

COM7021 MO Transcript.pdf

74.6 KB

Continue to 'Lesson introduction'

Lesson introduction

Welcome to Lesson 1

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

Welcome to Lesson 1 of the Data Visualisation module.

In this lesson, we will be introducing a multitude of visualisation elements and how vital the basics are in reporting and presentation environments. In Topic 1, we will discuss how visualisations impact society, the workplace and how visualisation can paint a very different picture to what numeric data may give. Topic 2 will be a shorter topic on the differences between graphics and visualisations, and will introduce Gestalt's Theory.

Topic 3 will recap basic chart plotting within Matplotlib and in Topic 4 we will focus on the introduction to basic chart plotting using Seaborn.



Computing

At Arden University

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

On completion of this lesson you will:

- know the basics of visualisation theory
- understand what perception is and the differences between graphics and visualisations
- gain further confidence in creating basic plot types in Matplotlib
- become familiar with advanced visualisations using Seaborn

Module learning outcomes

1. Critically appraise a variety of data visualisation techniques in terms of psychology, cultural context, effectiveness and target audience.
2. Apply appropriate data visualisation techniques to analyse and present data.
3. Demonstrate a critical understanding of how data visualisation can support technical and non-technical audiences in making business decisions.

Lesson 1 aligns to Learning Objective 3: **Demonstrate a critical understanding of how data visualisation can support technical and non-technical audiences in making business decisions**.



We have worked across teams at Arden University to try and create an accessible and inclusive experience within the learning content. We are continuously assessing the accessibility of our lessons to make improvements and enhancements based on testing and feedback.

This lesson may include screenshots of code and software used. Please be aware that these may not always have a text alternative, and that other features such as size and colour may not be accessible.

If you experience any issues with accessing the learning content throughout this module, there are several support services in place to help you.

- Arden University has a dedicated Inclusion Services Team who can help you with getting adjustments and equipment.
- The Library Portal also has information on the accessibility tools that can help you access resources: Accessibility Tools – Library Portal
- If you are struggling with accessing a specific part of the content e.g. a video resource that is missing captions, please speak to your Lecturer who may be able to advise on an alternative resource in the interim to ensure you are able to get the learning you need to progress with the lesson.



Continue to Topic 1

Topic 1: Introduction

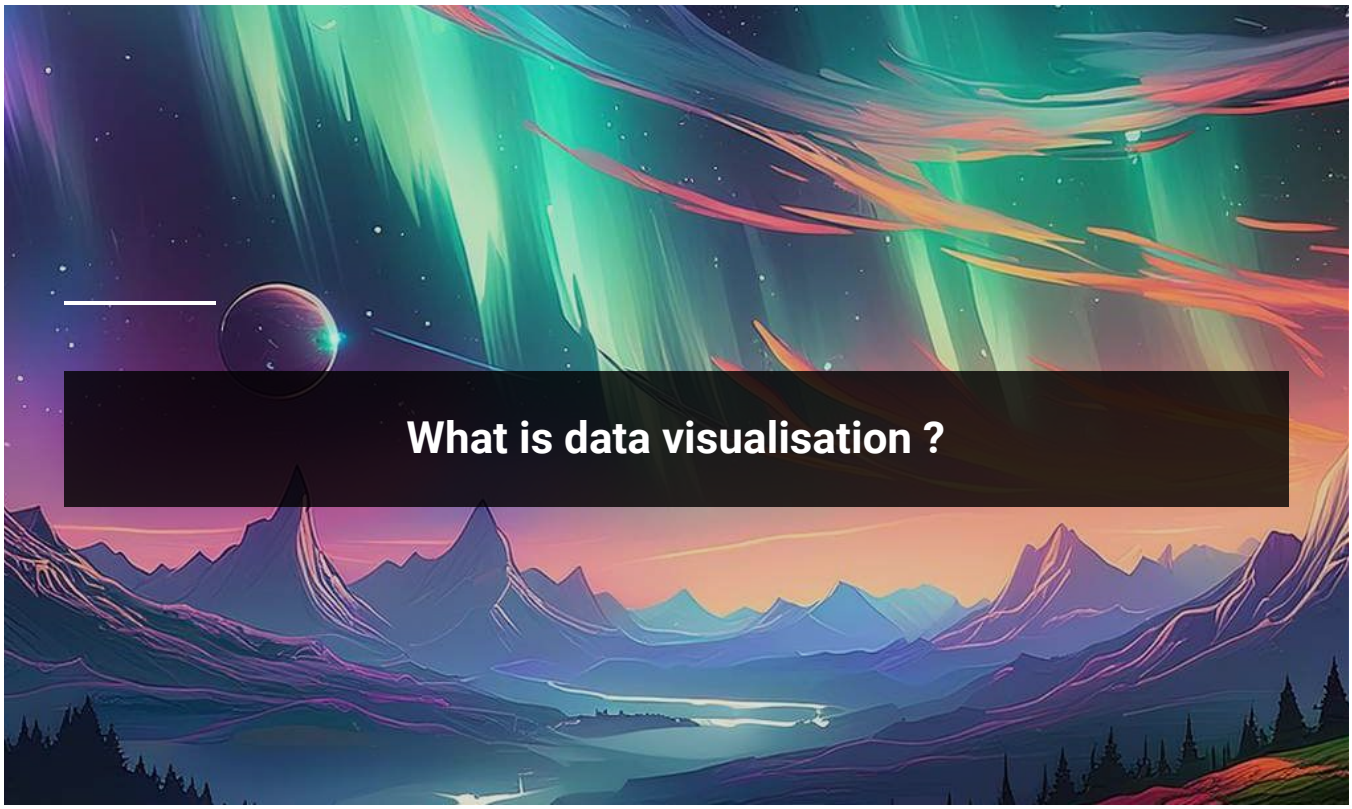


Image from Articulate Rise 360 Content Library.

We can define ‘visualisation’ as a form of graphical communication. Pictures have been used as a form of communication long before written communication. A picture, as I am sure you will agree, can contain a rich wealth of information, which can be processed by the human brain much more quickly than text, which is why we, as a species, have been using them for so long. We could therefore state that we are visual beings and few would probably argue with this. Another important item to recognise where pictures are concerned is that they are independent

of local language; in other words, a picture of a cat is still a picture of a cat irrelevant of local language.

In the modern, technological age in which we live today, the exponential gathering of data through mobile devices has increased the necessity of communication of both static and interactive data environments. Visual aids act as a medium by which viewers can observe information and ideas: requiring one or a series of two-dimensional images which can nowadays be designed using a multitude of different mediums, these include:

- typography
- graphic design
- industrial design
- signs
- drawing
- illustration advertising animation

Take the map of the London Underground for example; this map provides different levels of both qualitative and quantitative data. The map provides a relatively easy-to-use and read visualisation, although logically distorted, i.e. a crossing network. Within financial markets we hear quite frequently statements that the pound sterling (£) fell by 0.04 points against the US Dollar ($). A statement such as this provides a single piece of information. However, if we were to look at a plot of this it would give multiple pieces of information, such as time, rise and fall, and model data-driven projected trends. Therefore, the purpose of data visualisation today is to highlight patterns and trends that would possibly go unnoticed without the aid of such clear studies.

Continue



Image from Articulate Rise 360 Content Library.

If you are still not convinced on the importance of data visualisation, we are going to demonstrate how different sets of numerical data have the same basics statistics, yet when plotted on a graph, the visualisations tell a different story. Many of you may be familiar with the Statistician Francis Anscombe. If not, then you are encouraged to research his work. Anscombe developed the **Anscombe's quartet**, which consists of four data sets, each including 11 data points that have nearly identical simple descriptive statistics. The example code for the quartet and the visualisations are shown below in Figures 1.1 and 1.2.

```
[1] import matplotlib.pyplot as plt
    import numpy as np
```

```
x = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
y1 = [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68]
y2 = [9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74]
y3 = [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73]
x4 = [8, 8, 8, 8, 8, 8, 8, 19, 8, 8, 8]
y4 = [6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89]

datasets = {
    'A': (x, y1),
    'B': (x, y2),
    'C': (x, y3),
    'D': (x4, y4)
}
```

```
fig, axs = plt.subplots(2, 2, sharex=True, sharey=True, figsize=(6, 6),
                        gridspec_kw={'wspace': 0.08, 'hspace': 0.08})
axs[0, 0].set(xlim=(0, 20), ylim=(2, 14))
axs[0, 0].set(xticks=(0, 10, 20), yticks=(4, 8, 12))

for ax, (label, (x, y)) in zip(axs.flat, datasets.items()):
    ax.text(0.1, 0.9, label, fontsize=20, transform=ax.transAxes, va='top')
    ax.tick_params(direction='in', top=True, right=True)
    ax.plot(x, y, '*')

    # linear regression
    p1, p0 = np.polyfit(x, y, deg=1)  # slope, intercept
    ax.axline(xy1=(0, p0), slope=p1, color='r', lw=2)

    # add text box for the statistics
    stats = (f'$\\mu$ = {np.mean(y):.2f}\n'
             f'$\\sigma$ = {np.std(y):.2f}\n'
             f'$r$ = {np.corrcoef(x, y)[0][1]:.2f}')
    bbox = dict(boxstyle='round', fc='blanchedalmond', ec='orange', alpha=0.5)
    ax.text(0.95, 0.07, stats, fontsize=9, bbox=bbox,
            transform=ax.transAxes, horizontalalignment='right')

plt.show()
```

Figure 1.1. Example code for Anscombe's quartet, by Arden University. (Accessible code).

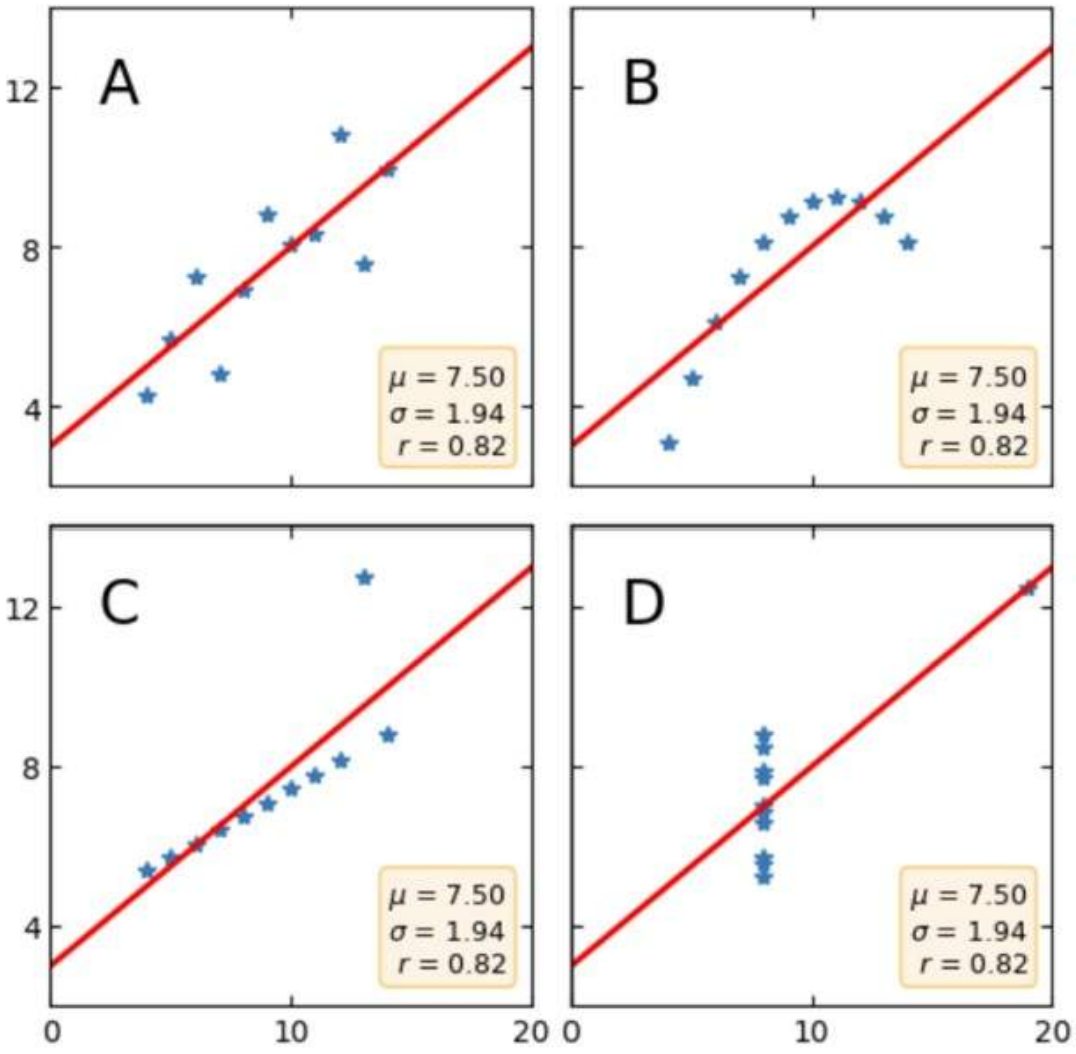


Figure 1.2. Anscombe's quartet visualisation, by Arden University. (Accessible code).

The first scatterplot **A** (top left) appears to be a simple linear relationship result, where two corresponding variables appear to be correlated, following the assumption that the data is normally distributed.

The second graph **B** (top right) is not distributed normally; while a relationship between the two variables is there, it is obviously not linear.

In the third graph **C** (bottom left), the distribution is linear, but should have a different regression line. The calculated regression is offset by the one outlier, which would exert enough influence to lower the correlation coefficient.

The fourth graph **D** (bottom right) shows an example when one outlier is enough to produce a high correlation coefficient, even though the other data points do not indicate any relationship between the variables.



Image from Articulate Rise 360 Content Library.

Graphical representation is not only limited to charts and plots. With the increasing amount of computer power at our fingertips, more complex problems can now be visualised in a manner not achieved before by a data scientist. Take for example the air flow over an airplane's wings, the downforce exerted on a Formula 1 car, or the way in which a boat's hull can be designed to produce the least amount of surface drag across the sea. Opportunities for generated simulations and visualisation are increasingly only limited to our imagination. There is an important question which must also be addressed, 'What is the difference between visualisations and computer graphics, if any?'

Continue

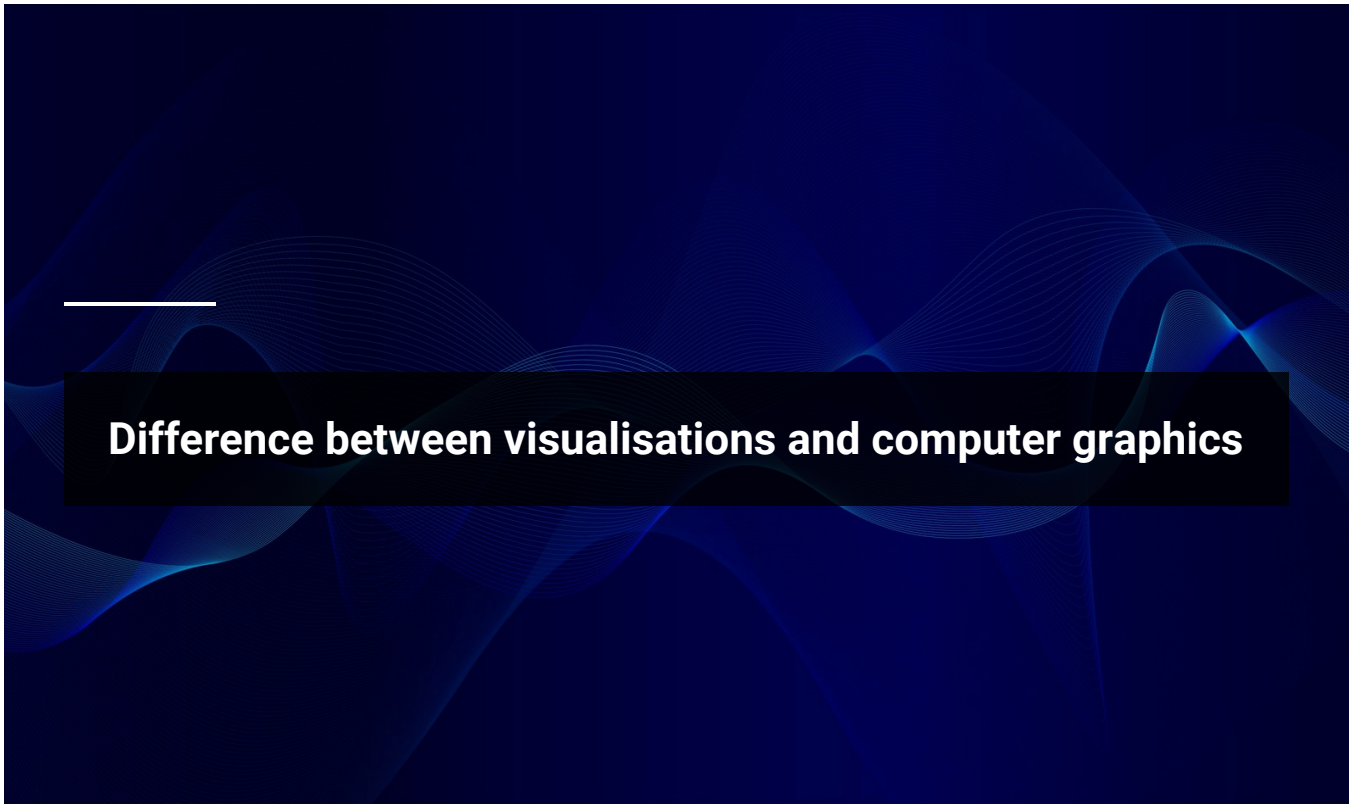


Image from Articulate Rise 360 Content Library.

In the very early stages of computer graphics, visualisations were regarded as a sub-field, because visualisations used graphics to give information through images. However, visualisations are much more than simplistic computer graphics and as such encompass other fields such as Human Computer Interaction (HCI), databases and statistics to name but a few.

Computer graphics is primarily used within the creation of interactive images and 3D animations, in addition to art, video games and visual realism. Visualisations on the other hand tend not to focus on visual realism or physical objects. So, although computer graphics and visualisations share many common concepts and techniques, their goals and models are fundamentally quite different. Hopefully, the visualisation pipelines of both shown below in Figure 1.3 will help distinguish the processes between them.

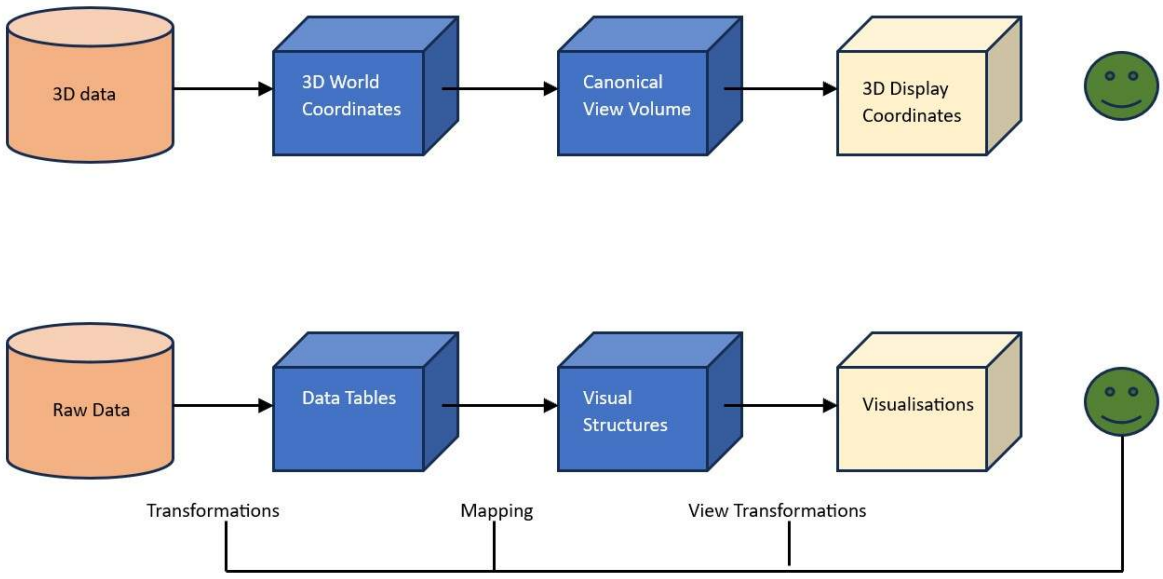


Figure 1.3. Pipeline process for graphics (top) and visualisations (bottom), by Arden University. (Accessible text).

In addition to the pipelines detailed in Figure 1.3 above, the knowledge discovery pipeline, i.e. data mining, has its own pipeline as shown below in Figure 1.4. In this particular pipeline, there is considerable focus on the data. There is also a lot of focus on the various data sets from multiple, or single locations that are to be analysed, which would include filtering and aggregating, in other words data integration and cleaning. Once the analysis has completed the first two items mentioned, data mining will then take place, which is at the heart of the knowledge discovery pipeline. The models are then evaluated and visualisation shown. The knowledge discovery pipeline is shown below as a comparison.

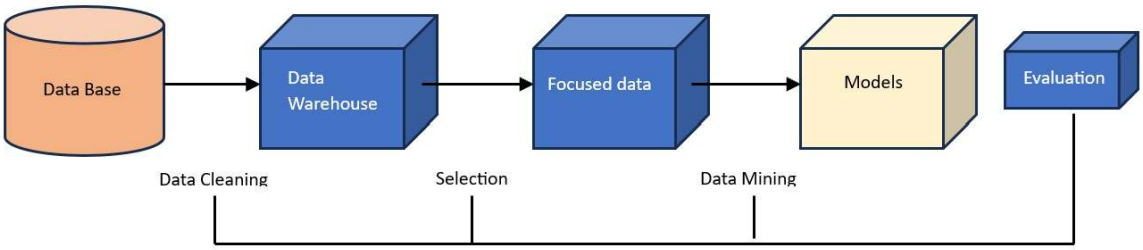


Figure 1.4. Data mining pipeline, by Arden University. (Accessible list).

Continue





Image from Articulate Rise 360 Content Library.

If the main aim of visualisations is to accurately give information in picture format, it is very important that human perception is always considered. A well-presented visualisation can be very appealing, but if we were presenting a conclusion, then we want to avoid ambiguity. The human brain will receive data and process it in many ways. One such process is **pre-attentive** processing: this particular human process is very fast and quickly identifies differences in colour and texture. In Figure 1.5 below, there is a colour distractor, which can be easily recognised amongst all others.

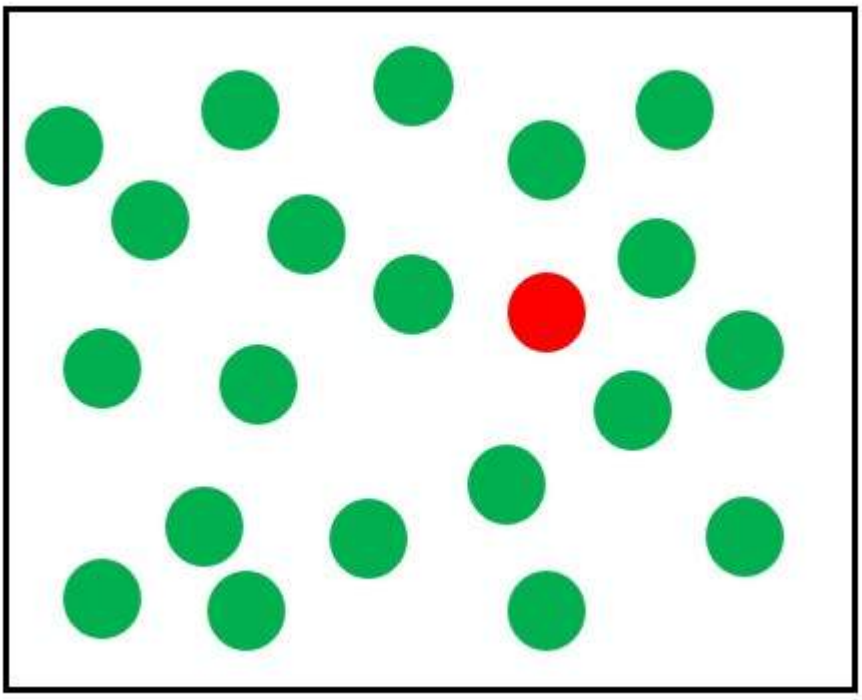


Figure 1.5. The graphical representation of a colour distractor, by Arden University.

In Figure 1.6 below, we demonstrate another distractor, but instead of a colour distractor we have a target distractor. In other words, it is the line orientation which is the perceptual element.

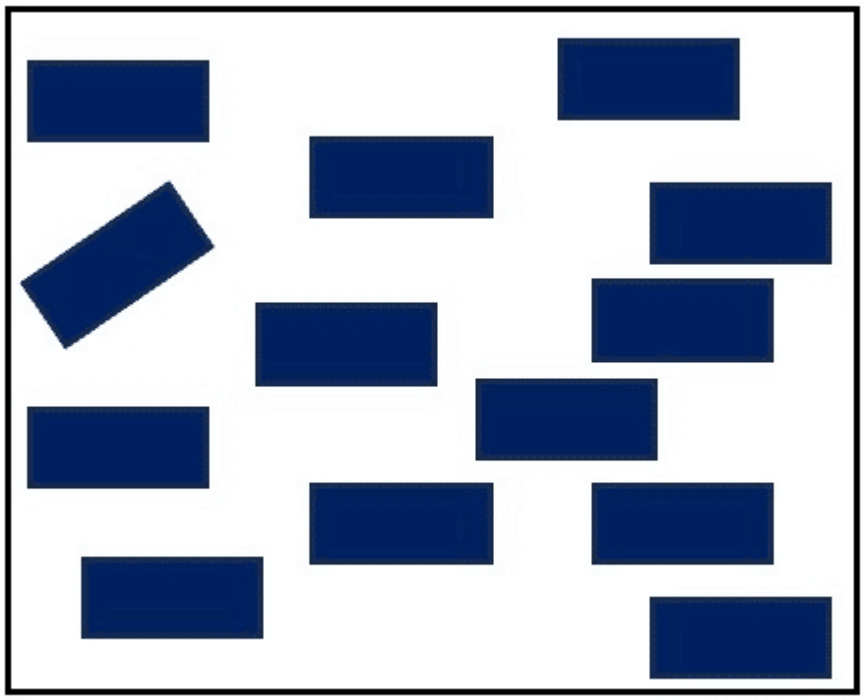


Figure 1.6. Graphical representation of an orientation distractor, by Arden University.

Learning about perception inherently comes with some guidelines and these can be explained within the Gestalt Theory.

Continue



Image from Articulate Rise 360 Content Library.

As we alluded to previously, our vision is essential to pattern recognition, therefore the simpler we can create visual material, the easier it is for our brains to make sense of the data. When information is seen in the context of other data sets within the same context, then that data can look very different than how it is perceived on its own: so, making relationships and connections as visually obvious as possible can be very advantageous, which leads us nicely into Gestalt principles for visual storytelling.

The Gestalt principles describe how our mind organises individual visual elements into groups. We do this to make sense of the world in which we live. When you go on to design your data visualisations, you can employ these principles to highlight patterns that are important within

your data sets. Gestalt's visual principles can be seen in Figure 1.7 below and a brief explanation for each principle is also given.

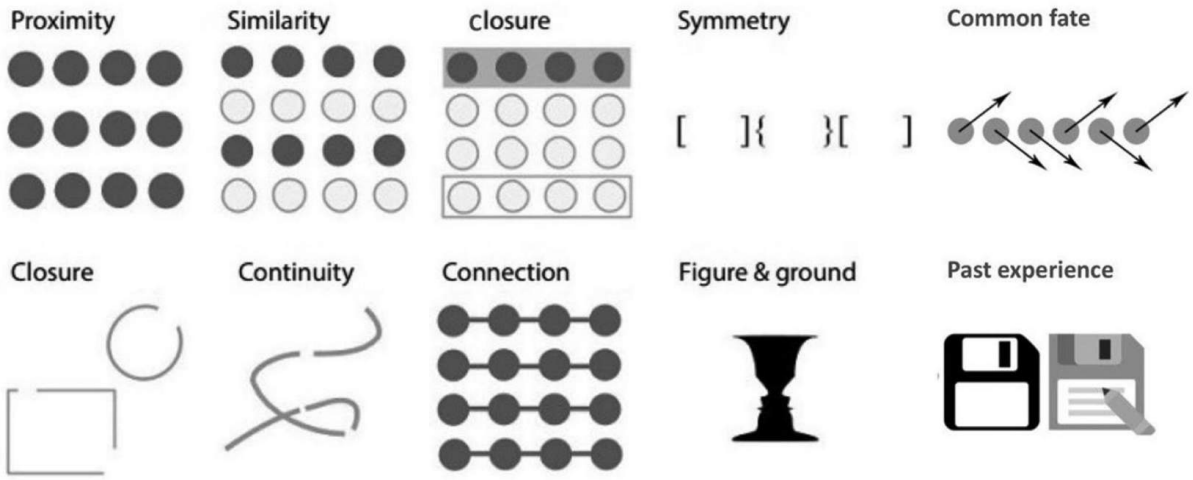


Figure 1.7. Gestalt's visual principles (Yalcinkaya and Singh, 2019, p. 528)

- **Proximity:** We see three rows of dots instead of four columns of dots because they are closer horizontally than vertically.
- **Similarity:** We see similar-looking objects as part of the same group. Enclosure: We group the first four and the last four dots as two rows, instead of eight dots.
- **Symmetry:** We see three pairs of symmetrical brackets rather than six individual brackets.
- **Closure:** We automatically close the square and circle instead of seeing three disconnected paths.
- **Continuity:** We see one continuous path instead of three arbitrary ones.
- **Connection:** We group the connected dots as belonging to the same group. Figure and ground: We either notice the two faces, or the vase. Whichever we

notice becomes the figure, and the other the ground.

- **Common fate:** Objects moving in the same direction appear to belong together, as a single unit.
- **Past experiences:** Our past experiences influence our future visual perception.

Perception is a visual interpretation which is something you have maybe not noticed before in the wider world, but hopefully now that we have highlighted it and stated its importance, it will give some food for thought when creating your data visualisation in the future.

Continue

Topic 1 - Activity 1

Anscomb's Quartet



Import the Anscomb's Quartet data and conduct the visual representation of the data set, plus investigate the descriptive statistics for yourselves.

Write a brief report on your findings and post them in the group's Discussion Forum for further critical analysis by your peers and tutor.

XLS **Anscomb's Quartet.xlsx**
34.1 KB

Continue

Topic 1 - Activity 2

Gestalt's principles



Further research Gestalt's principles and expand on the bullet points made in the lesson on this topic. Once you have concluded your research, write a brief report expanding on each point. The brief report is to include why each principle is important, plus give context within a data visualisation study.

Post your report in the group's Discussion Forum so further comments can be made by your peers and tutor.

Continue

Topic 1 - Activity 3

Reading



- Chapter 1: Introduction (pp. 22-50) in *Interactive Data Visualisation* by Ward, Grinstien and Keim

- Chapter 1: Data, Stats and Stories – An Introduction (pp. 39-70) in *Statistics and Data Visualisation with Python* by Rogel-Salazar
- **Wider reading:** Chapter 1: Becoming Visual (pp. 12-35) in *Data Visualisation made Simple* by Sosulski

Continue

Important point

When conducting any data investigation, reading the data alone cannot only be difficult, but it can be misleading. Therefore, it is vitally important to visualise the data to help guide you in further analysis.

Continue

TOPIC REVIEW

Well done, you have completed Topic 1.

This topic has given an introduction to what data visualisation is and how it is vital in report and presentation environments, especially within the workplace.

Hopefully, the initial part of this topic has given you the opportunity to look at how visualisations impact society and how influential they can be. It must be noted, however, that visualisation can paint a very different picture to what the numeric data may give and Anscombe's quartet, which we gave an example of, is testimony to this.

The initial topic is now complete so we can progress to possibly another item that could be overlooked when dealing with visualisations, perception.

Continue to Topic 2

Topic 2: Common visualisation mistakes



Image from Articulate Rise 360 Content Library.

It is common that new data visualisation creators create bad data visualisations and in the most part this is because they have never been formally educated in the topic: new creators make poor design choices which can have a detrimental effect on both data visibility and communication, which in turn can have a devastating effect on company resources, which will be addressed further on in this topic. To help avoid this, we shall now discuss bad data visualisation examples and common best practices.

Learning how to make and present compelling data visualisations is no longer a choice, t's a necessity. That being said, let’s understand the implications of bad data visualisations from an organisational perspective.

Waste of valuable resources: When data visualisations are unclear, stakeholders may spend additional time and effort reviewing, questioning and seeking clarification. This prolonged review process consumes valuable time and resources, causing delays in decision-making and operational inefficiencies.

User disengagement: Humans have an increasingly short attention span. Therefore, there is only a small window of opportunity for a data visualisation to make its point. If a data story, which we shall discuss at length further on in the module, lacks clear, understandable visualisations, stakeholders will lose interest relatively quickly. As such, the disengagement has a tendency to hinder important follow-ups and discussions.

Confused audience: The ultimate goal of data visualisation is to make insights clearly understandable. However, when they showcase inaccurate data or complicated visuals it can result in misinterpretation, which in turn breeds even more confusion and inaction.

Continue

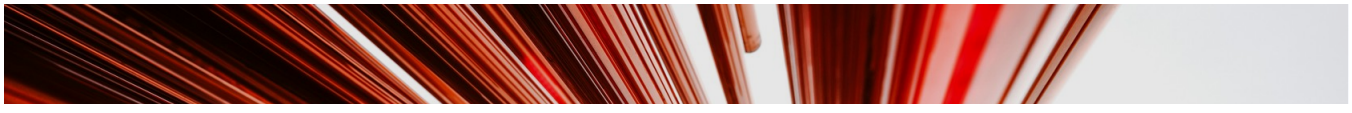




Image from Articulate Rise 360 Content Library.

Even at this early stage in the module, I hope we can now at least determine that data visualisation is not solely concerned with converting information into mesmerising charts. It involves combining a certain aesthetic element of design with analytical and methodical aspects. Using the wrong type of chart is often misleading and can easily make visualisations difficult to digest.

In the example below, shown in Figure 1.8, we have given an example of a misleading data visualisation or the same data set. As we shall see in subsequent lessons, histograms and bar charts help consumers to understand data from a distributional aspect. If we were to instead use a pie chart for data that is distributed, it could lead to a lack of clarity in the data, therefore making it harder for the consumer to understand the spread and variability contained within it.

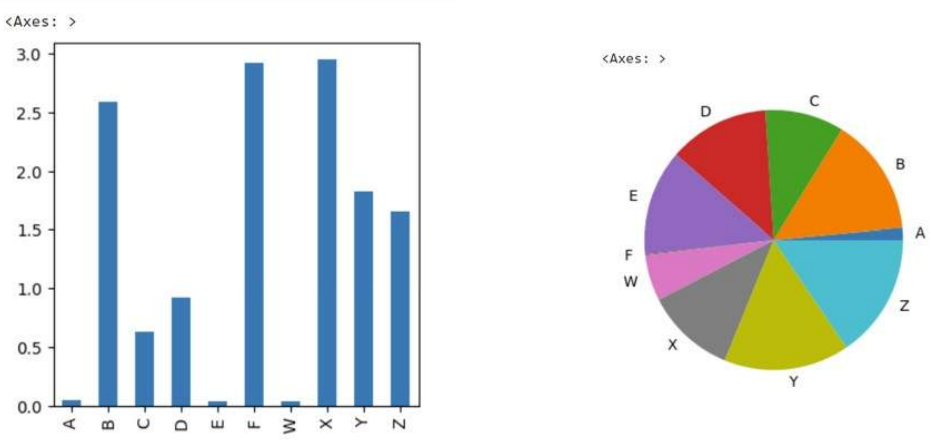


Figure 1.8. Comparative data visualisation where data clarity can be lost due to the wrong choice in plot, by Arden University.

Continue

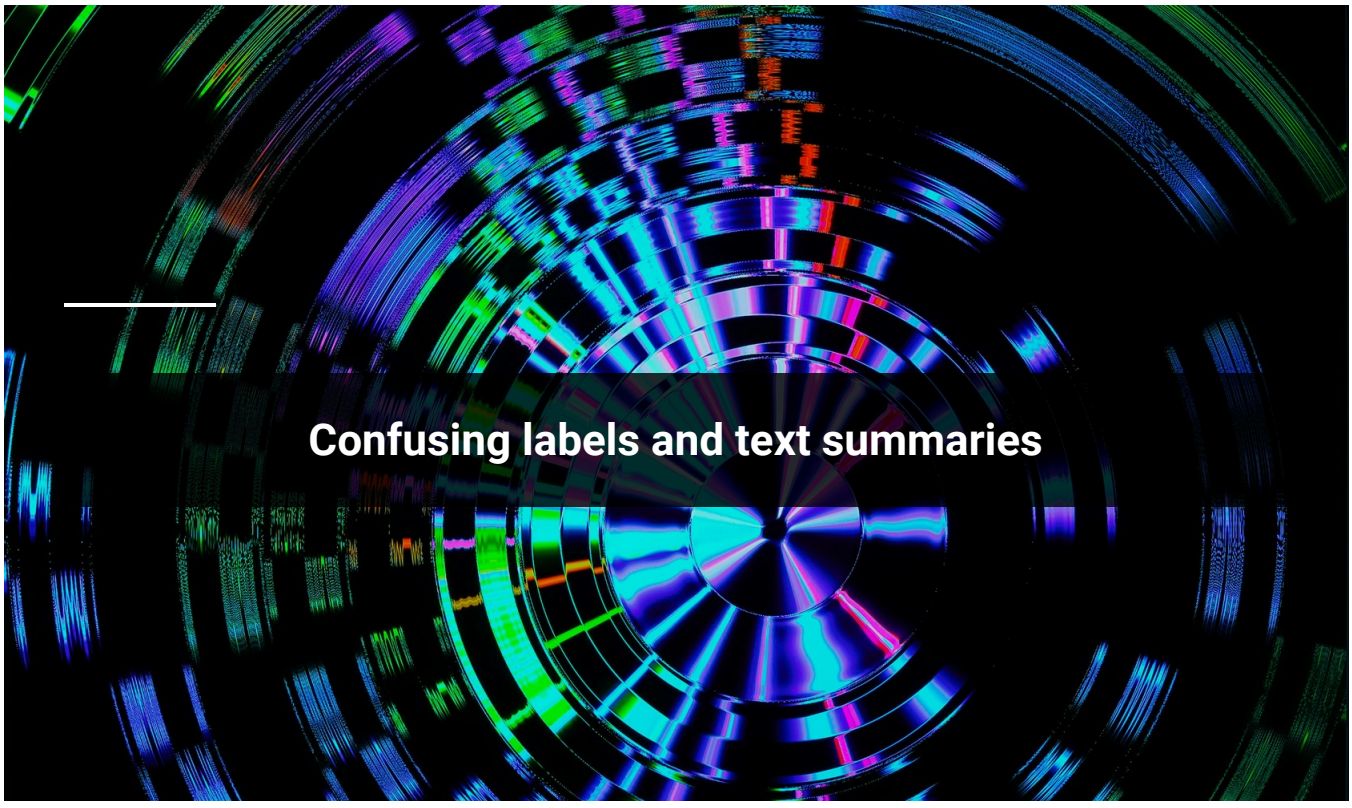


Image from Articulate Rise 360 Content Library.

Without data context, visualisations hold very little to no value. With relevant summaries, labels and descriptive fields, the data analyst can deliver background information and facts surrounding a dataset to the consumer, thus enabling more informed, data-driven decisions. However, context needs to be added to highlight critical points and to ensure that misleading summaries are kept to a minimum. However, this is easier said than done for those new to data representation through visualisation.

In most cases, the analyst can add context by adding percentages to pie and bar charts, so that increases/decreases can be easily understood. Small inclusions like this can help decision-makers decide company and departmental direction.

Additionally, making these small but significant inclusions can enhance clarity or reinforce vital points in the data. The analyst can also add additional indicators to the visualisation to further explain important trends or outliers within the data and ensure that textual summaries are simple and easy to understand.

Continue



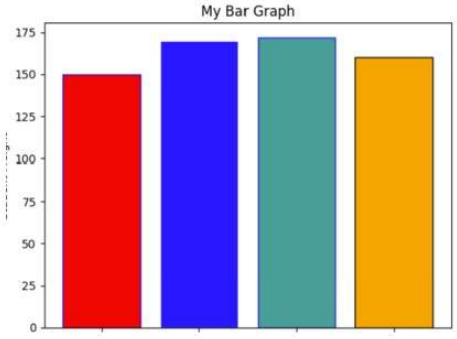


An inconsistent colour palette

Image from Articulate Rise 360 Content Library.

When colours are used in a contextual manner, they can have a strong effect to any visualisation and can help highlight important findings. Colour can easily communicate data insights, establish relationships between two distinct objects in the visualisations and remove visual overload to the consumer. Inconsistent colours will not help a consumer understand the data more, in fact, it could have the opposite effect.

In Figure 2 below, a bad data visualisation example is shown, highlighting the significance of using colours strategically. The bad example would be on the right of Figure 1.9.



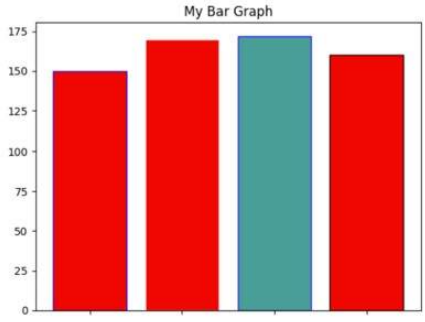


Figure 1.9. Comparative example of a good and bad colour palette choice, by Arden University.

Continue



Image from Articulate Rise 360 Content Library.

As we shall see in subsequent lessons, creating a story with data visualisations is not only memorable, it adds impact. A common error to new visualisation analysts is that they believe that adding in as many facts as possible enhanced presentation: however, this is a mistake. Wherever possible, present the facts through engaging storytelling. In doing so, this will help the consumer to understand the motivations and better retain important insights. In other words, when telling a data story, connect the dots and don't let them be singular points of information without context. One of the most important ways in which to facilitate action from the consumer is to introduce the problem, describe the data-driven visualisation and seamlessly

unpack your results. So, in essence, you are creating, through visualisation, a beginning, middle and an end, with a couple of contextual real-world examples thrown in.

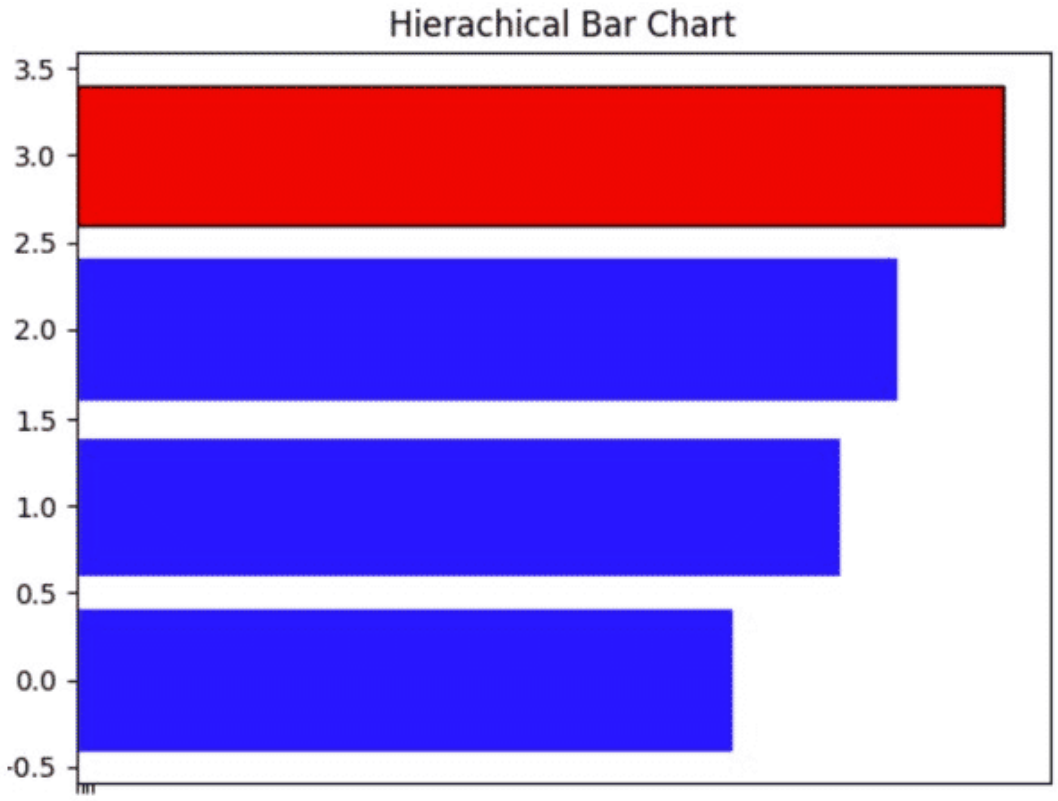


Figure 1.10. Hierarchical horizontal bar chart drawing emphasis on the higher value, by Arden University.

Data visualisations can create large movements in how organisational stakeholders make informed business decisions: this is because visual representation offers complex information to be accessed in a simple manner, bringing increased data insights. Some of the important inclusions included throughout this topic can be summarised below.

- Consider stakeholder preferences and expertise levels.
- Ask yourself if the chart serves its purpose.
- Emphasise clarity by highlighting vital points in the data.
- Add indicators.
- The colour palette should be simple yet effective.
- Limit the number of colours used to avoid unwarranted complexity.
- Do not overuse vivid colours.
- Keep consistency across visualisations.
- Your data story should include a beginning, middle and end.
- Include real-world examples or anecdotes.
- Logically order data elements that support hierarchy.
- Avoid unnecessary clutter.

Continue

Topic 2 - Activity 1

Reading



Chapter 2: Data Foundations (pp. 51-80) in *Interactive Data Visualisation* by Ward, Grinstien and Keim

Chapter 2: Python Programming Primer (pp. 71-136) in *Statistics and Data Visualisation with Python* by Rogel-Salazar

Wider reading: Chapter 2: The Tools (pp. 36-53) in *Data Visualisation made Simple* by Sosulski

Continue

TOPIC 2 SUMMARY

Well done, you have completed Topic 2.

Within Topic 2 we have looked at some of the common mistakes that can be made by new visualisation analysts. We have discussed using the wrong visuals, applying an inconsistent colour palette, plus the detrimental effect a lack of a narrative and visual hierarchy can have on a visualisation presentation. Many of the items included within Topic 2 will be further expanded upon as the module develops.

With the topic of common errors and proposed solutions now completed, we can move on to the visualisations themselves with a recap on Matplotlib in the following topic.

Continue to Topic 3

Topic 3: Basic visualisations using Matplotlib



Image from Articulate Rise 360 Content Library.

Analysing large volumes of data numerically is challenging, hence the importance of interpretating large data sets visually is a vital skill within any data scientists' theoretical tool bag, as you have no doubt determined from the previous two topics.

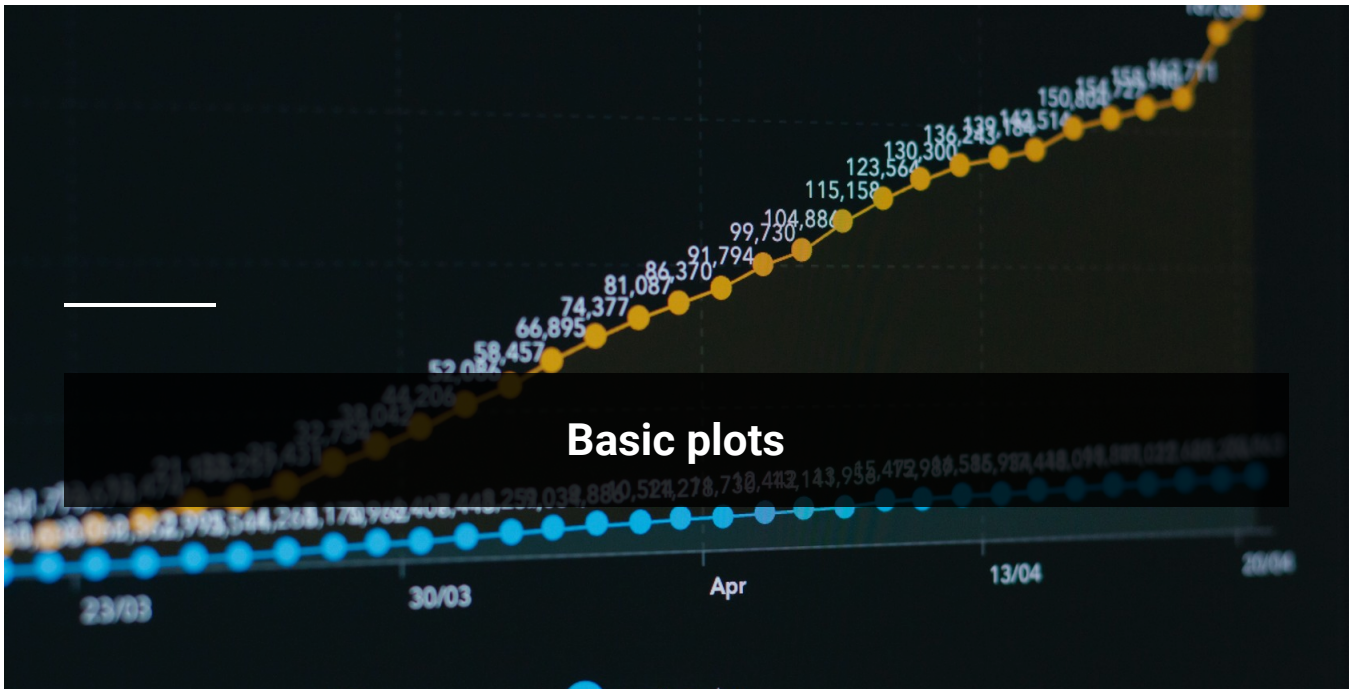
To address this visualisation need, Python has included a data visualisation library, although not the only one, so that analysts can visually represent the data in a simple, non-complex manner.

One of the primary libraries that analysts use within a Python coding environment is Matplotlib, which we were introduced to in an earlier module. The reason for this is that Matplotlib gives an easy way in which to interpretate data through various types of graphs and plots.

Matplotlib is an open-source library built upon Numpy arrays, which is why we took the time earlier on the program to take a deep dive into Numpy. Not only is Matplotlib designed to work with Numpy, but it seamlessly integrates with another Python library, namely scientific Python (SciPy). One of the big advantages to Matplotlib is that it has the capability of visualising very large data sets.

We shall now demonstrate through coded examples, as a recap, a multitude of charts reiterating how Matplotlib works and some of the plots it can produce. Engaging in a recap on basic plot types will hopefully cement the learning experience and allow additional confidence as we progress further in this lesson.

Continue



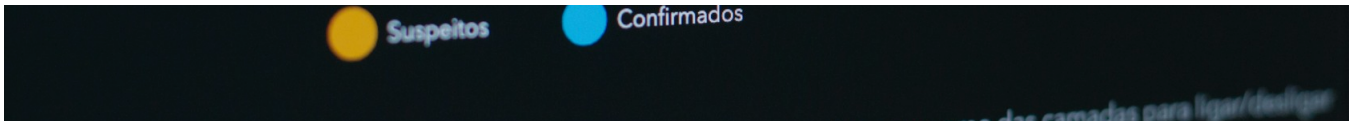


Image from Articulate Rise 360 Content Library.

We shall begin by randomly generating some data using the **np.random** command, which are seeded 123456. We shall then visually represent the randomly generated data using the **.plot()** command on a series, shown below in Figure 1.11.

Example

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
np.random.seed(123456)

bp = pd.Series(np.random.randn(500), index=pd.date_range("1/1/2000", periods=500))

bp = bp.cumsum()

bp.plot();
```

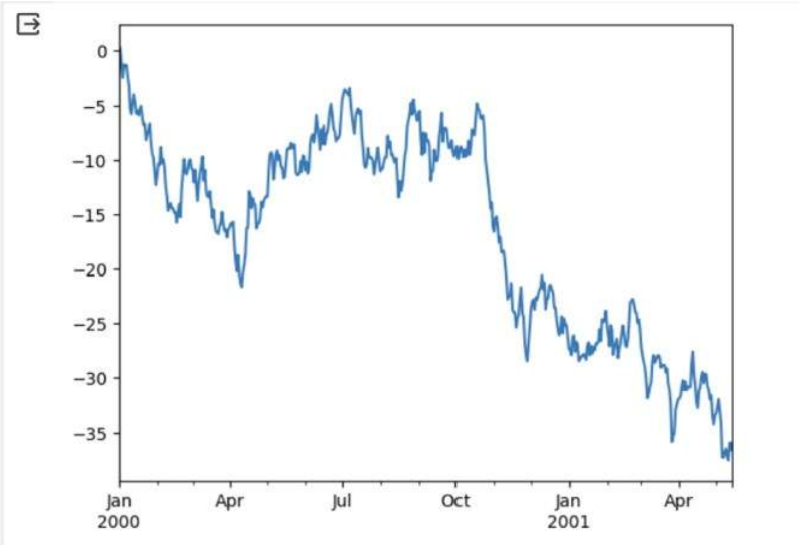


Figure 1.11. Simple line plot from randomly generated data, by Arden University. (Accessible code)

Where a dataframe is concerned, it is advantageous to plot all the columns with labels. In this case the labels are defined as W, X, Y and Z, as shown below in Figure 1.12.

Example

```
df = pd.DataFrame(np.random.randn(500, 4), index=bp.index, columns=list("WXYZ"))

df = df.cumsum()

plt.figure();

df.plot();
```

```
<Figure size 640x480 with 0 Axes>
```

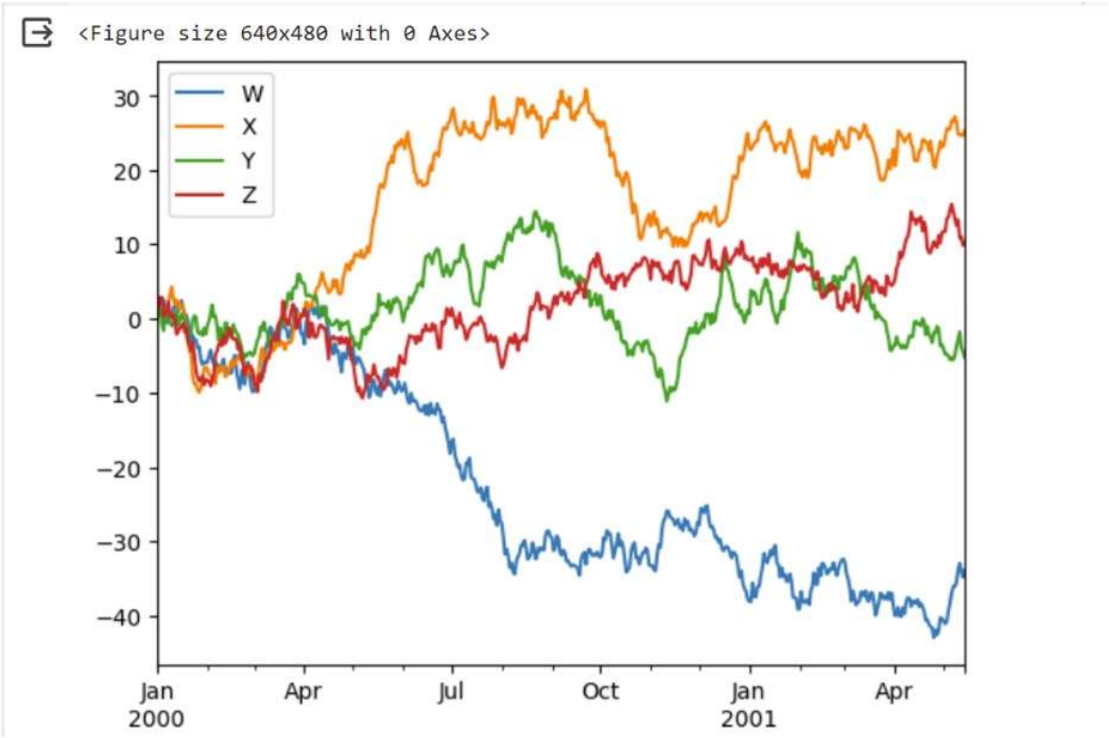


Figure 1.12. Multiple line plots from within the dataframe, by Arden University. (Accessible code).

In Figure 1.12 above, we have included all columns with the dataframe: this is very useful when trying to determine how multiple variables compare to each other over time, it is also very useful to focus in on the comparison of only two variables. This is shown below in Figure 1.13 using the X and Y keywords within **.plot()**.

Example

```
df3 = pd.DataFrame(np.random.randn(1000, 2), columns=["T", "U"]).cumsum()

df3["S"] = pd.Series(list(range(len(df))))

df3.plot(x="S", y="T");
```

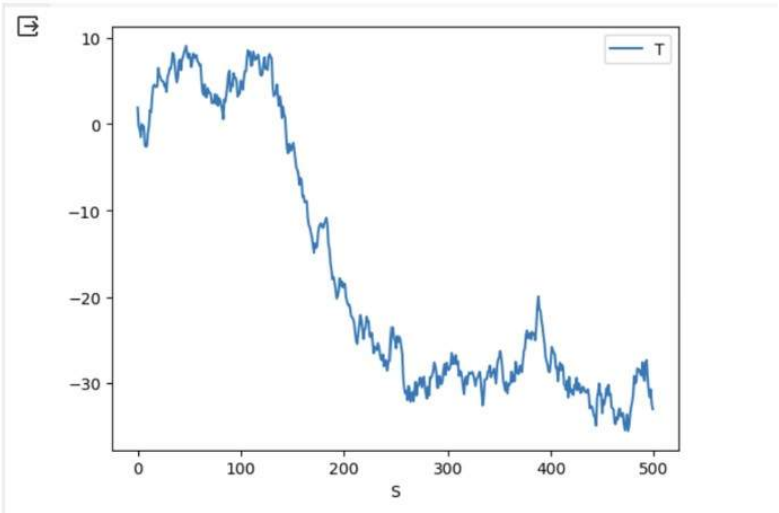


Figure 1.13. Comparative of variables over time and correlation, by Arden University. (Accessible code).

Continue



Image from Articulate Rise 360 Content Library.

Other plotting methods are available within Matplotlib in addition to the basic line plot we have demonstrated above. These additional methods can be used using the kind keyword within the **.plot()** command. Some of these keywords are pretty intuitive, such as 'bar' for bar charts, 'hist' for histogram, 'box' for box plots, 'density' for density plots, 'scatter' for scatter plots and 'pie' for pie charts. Most of which will be demonstrated throughout this lesson. As a brief example, a bar chart can be created in the following way, as shown in Figure 1.14.

Example

```
plt.figure();

df.iloc[5].plot(kind="bar");
```

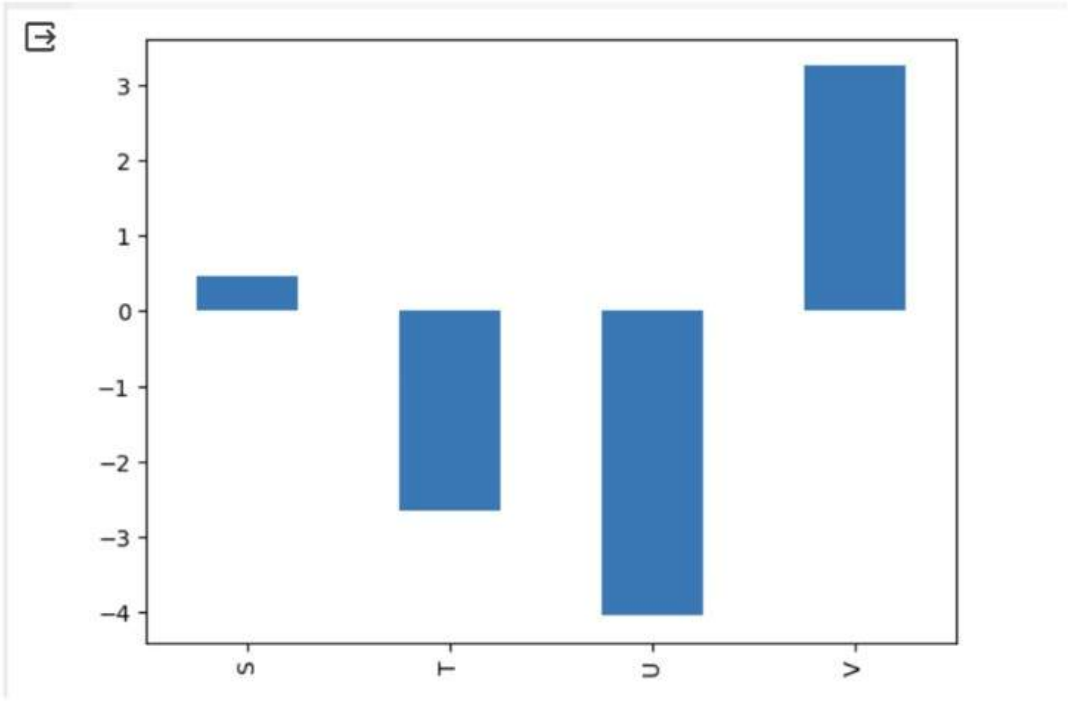


Figure 1.14. Comparative of variables over time and correlation, by Arden University. (Accessible code).

There is an alternative way, however, to using the kind keyword, which makes it easier to discover additional plotting methods when using the **dataframe.plot()** command. Examples of this simpler methods are shown below in Figure 1.15.

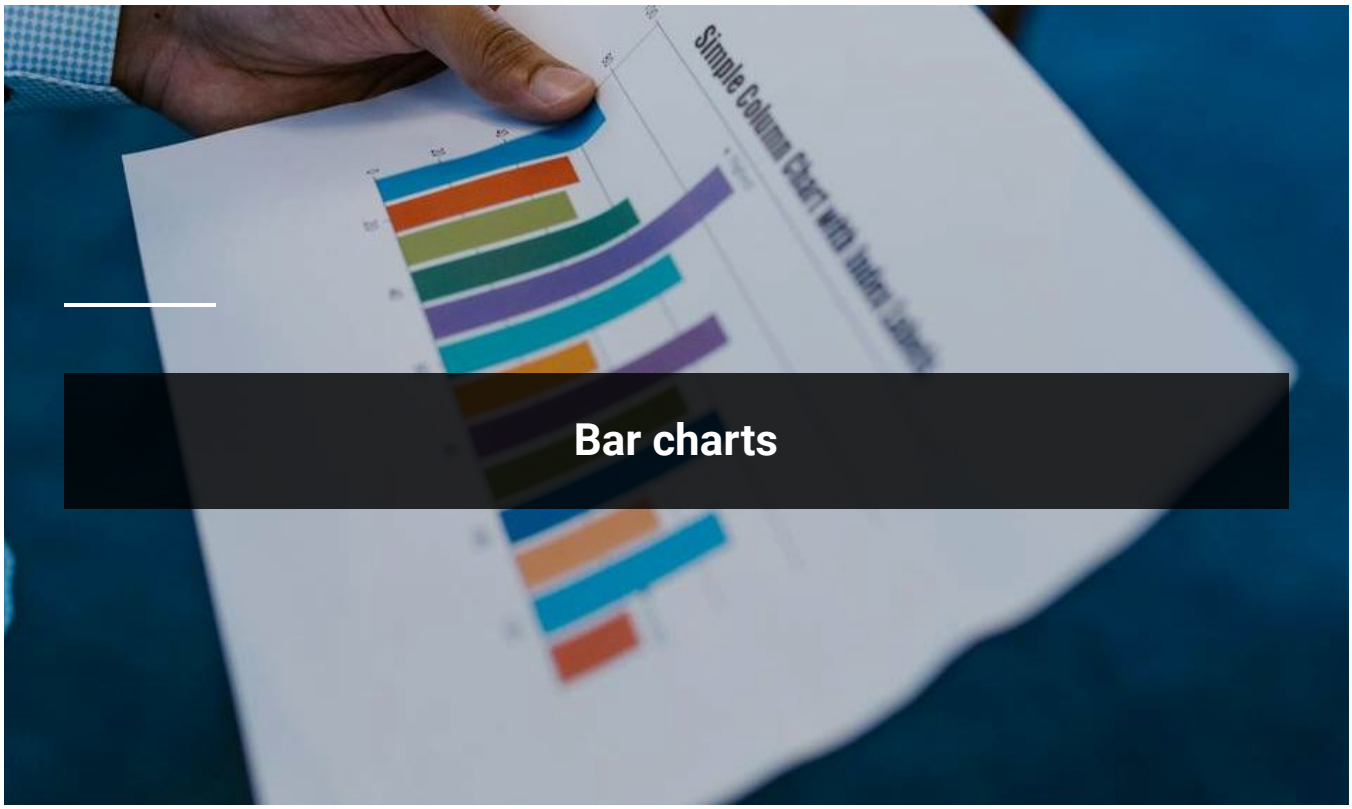
```
[ ]  df = pd.DataFrame()

     df.plot.<TAB>
     df.plot.area     df.plot.barh     df.plot.density  df.plot.hist     df.plot.line     df.plot.scatter
     df.plot.bar      df.plot.box      df.plot.hexbin   df.plot.kde      df.plot.pie
```

Figure 1.15. A list of alternative commands within the **dataframe.plot()** command, by Arden University. (Accessible code).

There are more advanced plotting functions within pandas which allows the user to take a series or dataframe as an argument, such as Andrew's curves, Lag plot, Autocorrection plot and Scatter matrix; however, we shall not be covering these in this module but have mentioned them for completeness.

Continue



Bar charts

Image from Articulate Rise 360 Content Library.

Bar charts are an ideal visualisation when you have labelled, non-time series data. In other words, when we have categorical, or discrete variables, such as the age of a group. Bar charts allow the user to see the comparison between variables with comparative ease, as shown below in Figure 1.16. However, caution must be applied with bar charts to ensure that not too many variables are included.

Example

```
plt.figure();

df.iloc[5].plot.bar(color="green");

plt.axhline(0, color="r");
```

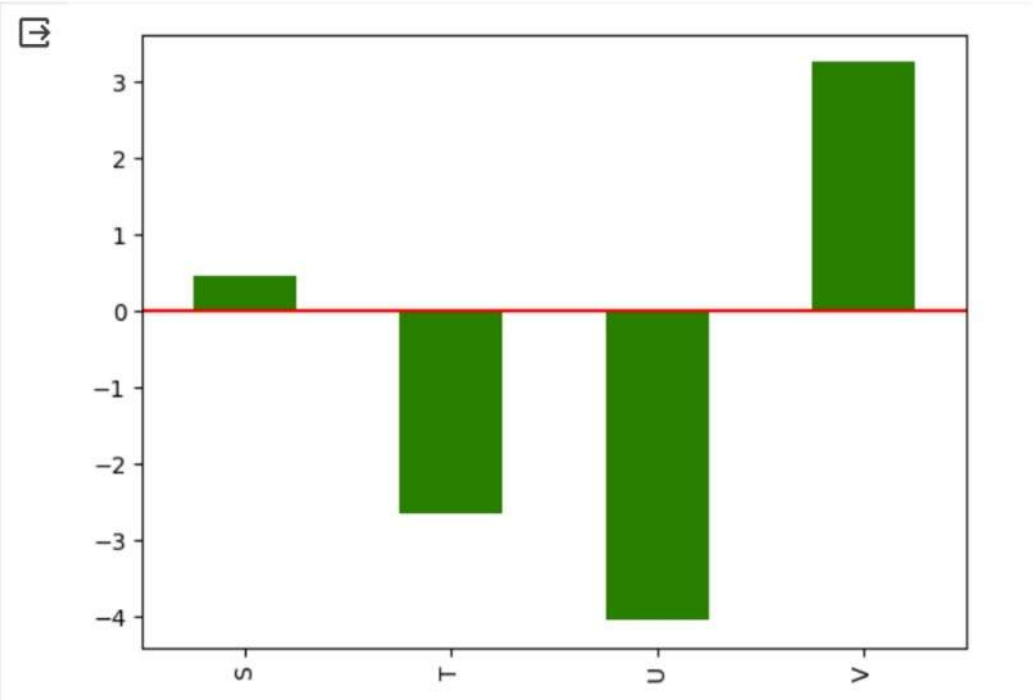


Figure 1.16. Simple bar chart where only a limited number of variables are used as comparisons, by Arden University. (Accessible code).

We can also use bar charts when comparing multiple variables using the **plot.bar()** command, as Figure 1.17 demonstrates.

Example

```
df2 = pd.DataFrame(np.random.rand(8, 3), columns=["W", "X", "Y"])

df2.plot.bar();
```

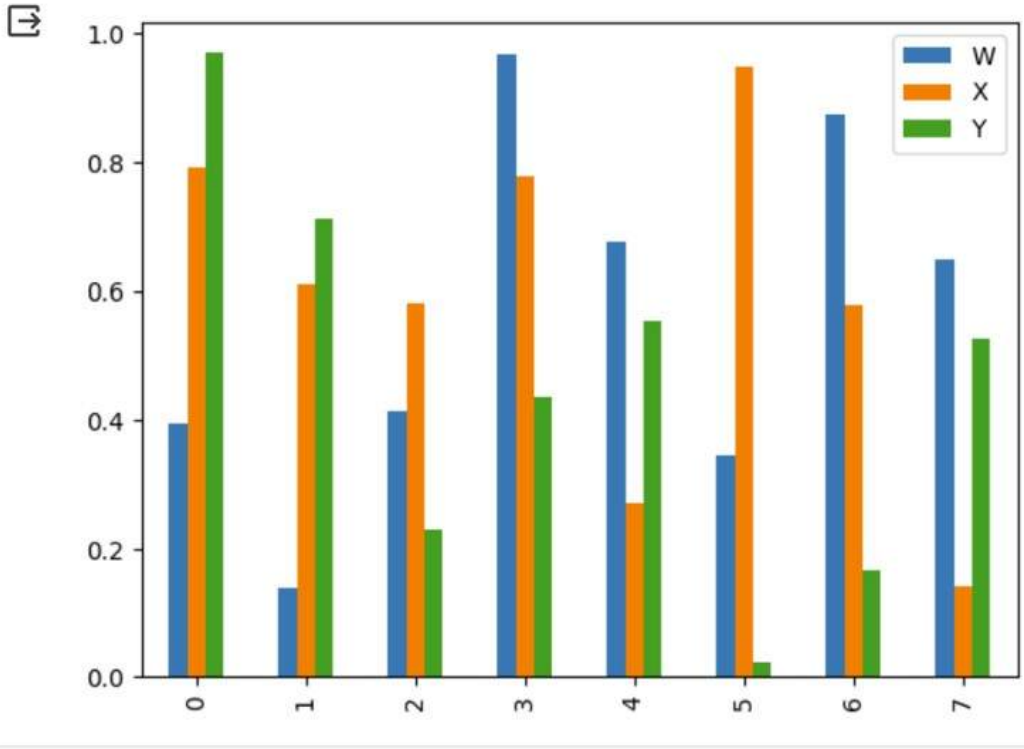


Figure 1.17. Identical variables compared across different data sets for a wider comparison, by Arden University. (Accessible code).

The analyst can also take advantage of staked bar charts: these particular bar charts are very useful when comparing numerical values across different variables. However, due to the comparative numerical variables, there are a limited number of scenarios when one would use them effectively. Ideally, stacked bar charts would be analysing numerical values, which have at least one or two categorical variables that can be broken down and include date intervals. However, used effectively they can be quite a powerful visualisation, as can be seen in Figure 1.18 below.

Example

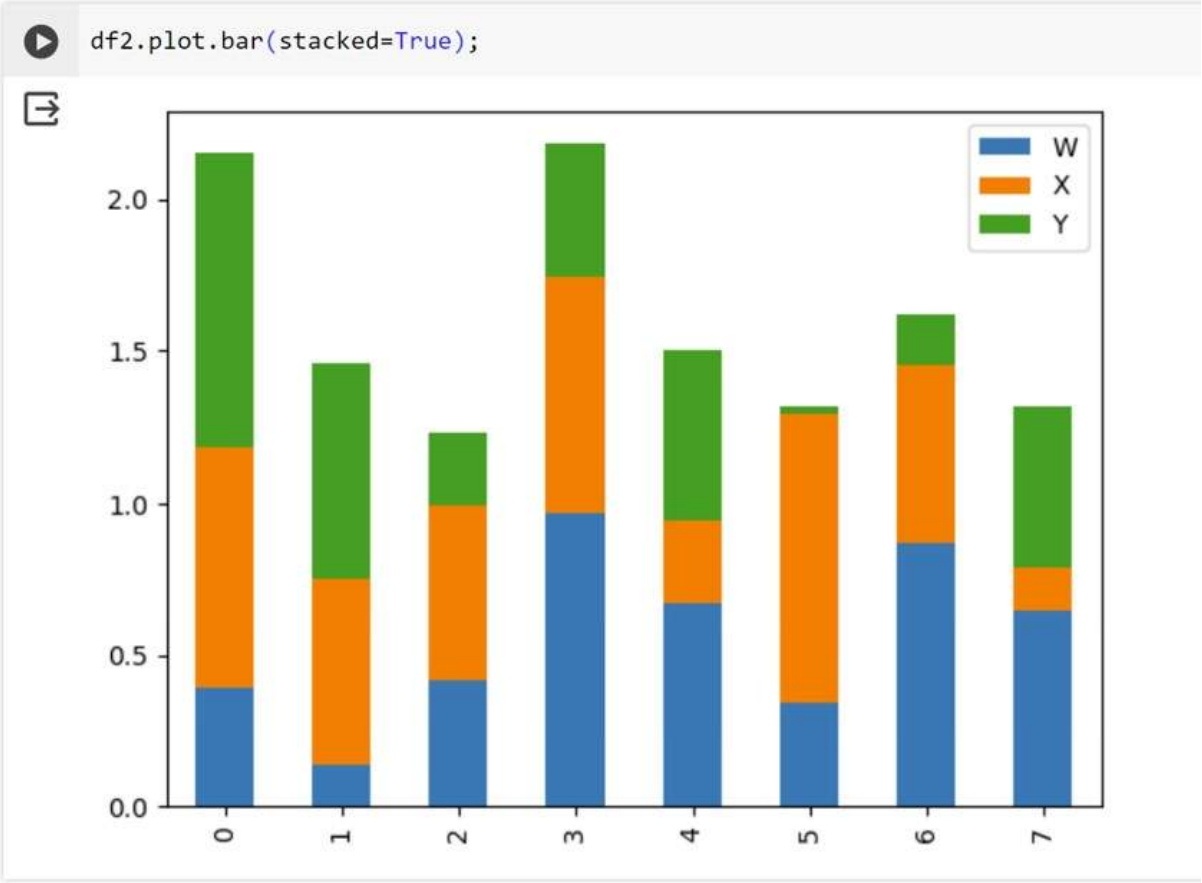


Figure 1.18. Stacked bar charts, by Arden University. (Accessible code).

Displaying bar charts is not only limited to vertical charts, although this the most common. Bar charts can also be represented horizontally, using the plot.barh function. Horizontal charts in some cases help the analysist to display the data in a more impactful manner, as Figure 1.19 demonstrates.

Example

```
df2.plot.barh(stacked=True);
```

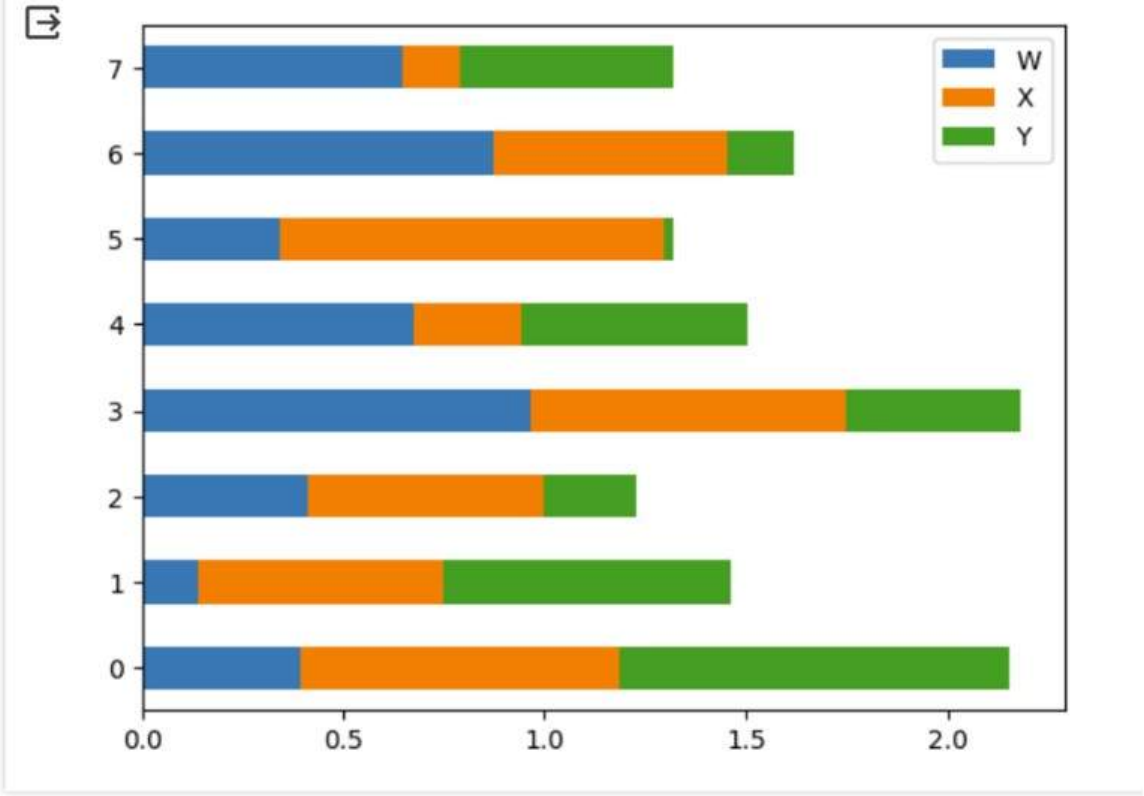


Figure 1.19. Horizontal bar chart to demonstrate alternative orientation, by Arden University. (Accessible code).

Continue



Image by Adobe Firefly.

As we shall see, histograms are very similar to bar charts, in so far as they look the same. However, histograms do not have a gap between the differing variable data like bar charts do. There is a very good reason for this and it is to do with the distribution of data within any given data set. Each group within the data set is divided into rectangles, usually of equal width and measure primarily comparative categorical data. So that we can easily determine the subtle differences between bar charts and histograms, we have constructed a simple table below.

Bar Chart	Frequency
Compares categories of data	Shows distribution of a variable

Bar Chart	Frequency
Categorical variables	Numerical variables
Data point represented as a single bar	Data is represented as groups based upon bin value.
Space between bars	No space
Possibility of re-ordering	No possibility of re-ordering

There are two ways in which we can represent variable distribution in Python using Matplotlib. We can either use the **dataframe.plot.hist()** command for a dataframe, or the **series.plot.hist()** for a series. For the following example, we shall only be demonstrating the dataframe option in Figure 1.20 below.

Example

```
import pandas as pd
import numpy as np


df3 = pd.DataFrame(
    {
        "W": np.random.randn(500) + 1,
        "X": np.random.randn(500),
        "Y": np.random.randn(500) - 1,
    },
    columns=["W", "X", "Y"],
)

df3.plot.hist(alpha=0.6);
```

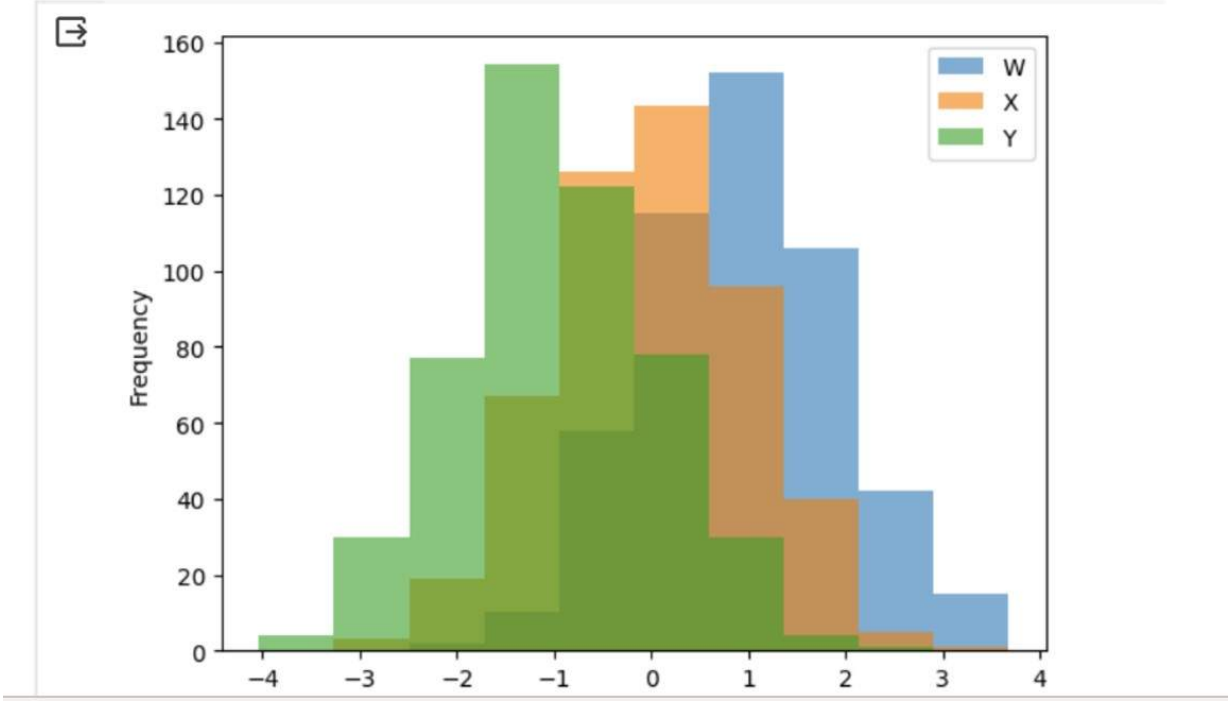


Figure 1.20. Basic histogram representing data variable distribution, by Arden University. (Accessible code).

One of the most important differences between histograms and bar charts is that histograms are set into bins and the bins are the groups, we, as the analysist, decide before the histogram is created. This is different to bar charts and in the following example we have determined the bins as 15, but we could have chosen 10, 12 or 11, but this will depend on the data set under investigation.

```
df3.plot.hist(stacked=True, bins=15);
```

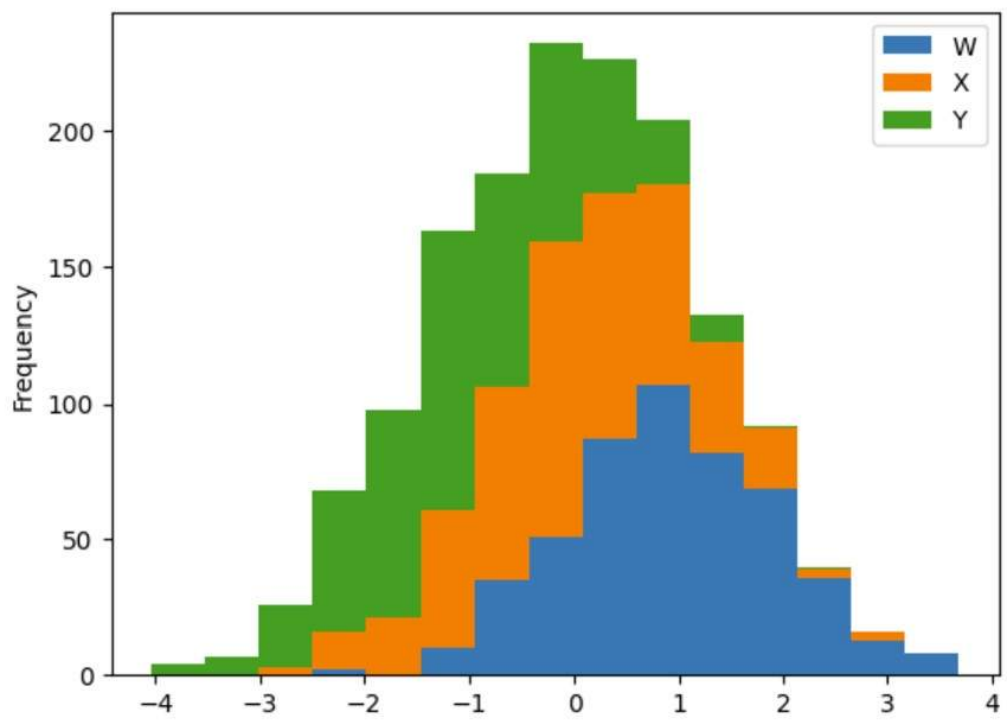


Figure 1.21. Histogram with pre-determined bins, by Arden University. (Accessible code).

There are other options which you can add into the **.plot.hist()** command for various histogram orientation, such as the orientation = histogram and cumulative = true commands, as shown in Figure 1.22 below.

Example

```
df3["W"].plot.hist(orientation="horizontal", cumulative=True);
```

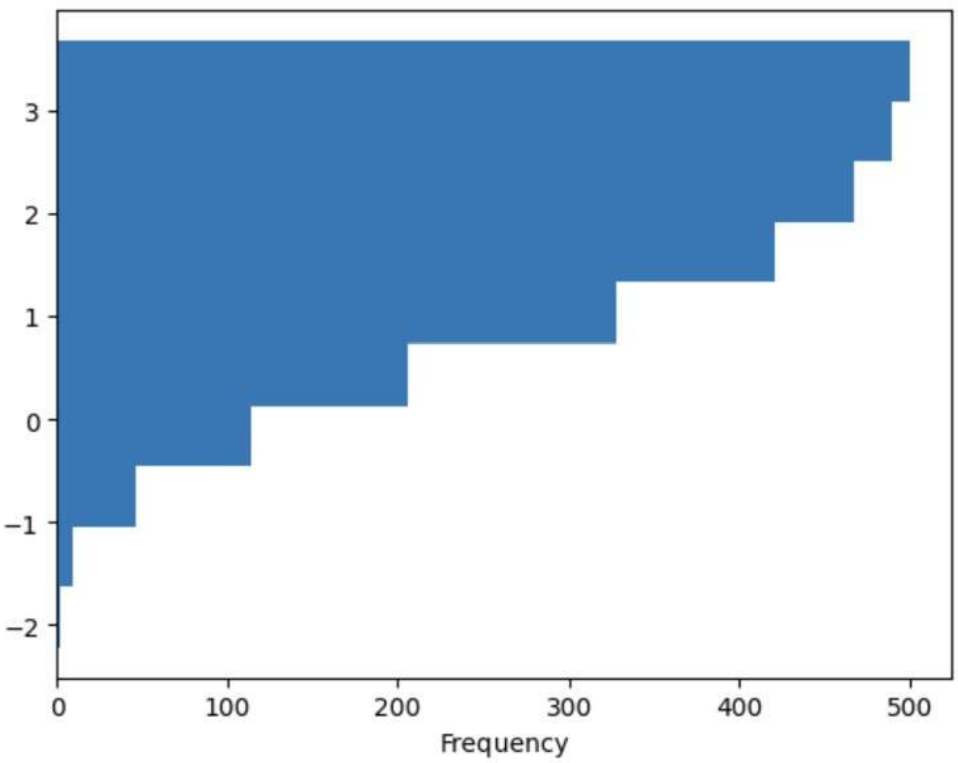


Figure 1.22. Different orientation of the histogram, by Arden University. (Accessible code).

There is also the option to display all columns within subplots, which is shown in Figure 1.23 below.

Example

```
df3.diff().hist(color="r", alpha=0.7, bins=30);
```

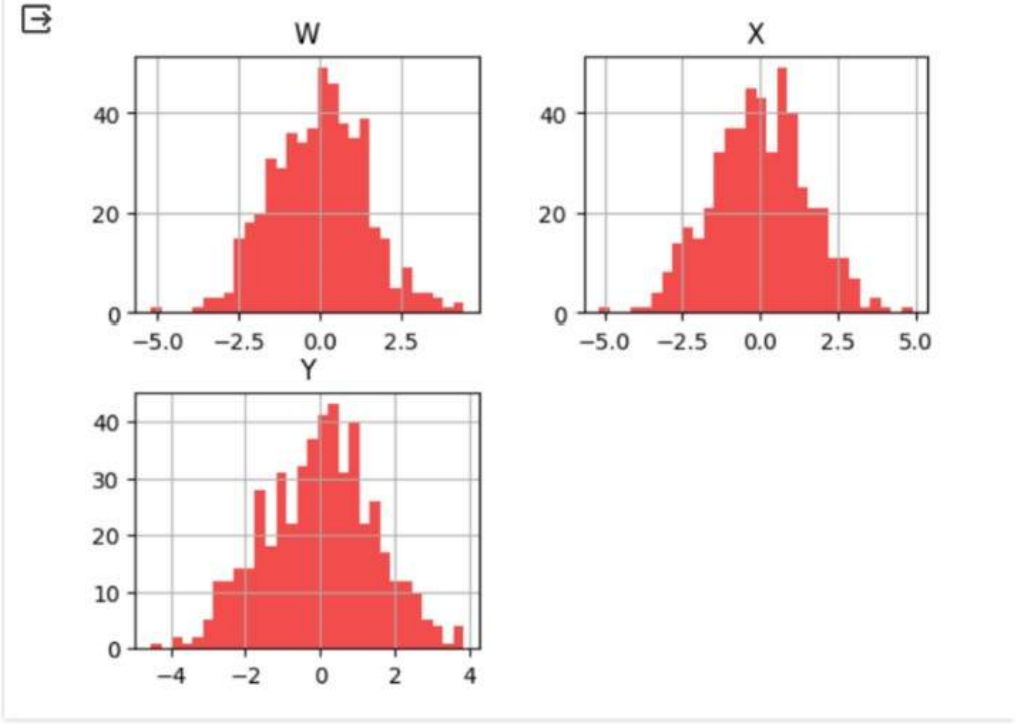


Figure 1.23. Displaying all columns within the data set as subplots, by Arden University. (Accessible code).

Continue





Image from Articulate Rise 360 Content Library.

A box plot, often referred to as a box and whisker plot, is an ideal visualisation to use when we wish to show the distribution of the data points across a selected measure, including median, mode, minimum, maximum and most importantly, outliers. Most of the data points, at least 50%, will lie within the box itself. The box will show the upper and lower quartile of the data, with the median in the middle. The lower quartile is deemed the 25th percentile, while the upper quartile is referred to as the 75th percentile. This would allow the analyst to determine the spread of data across one particular variable when compared across many different subjects. Take footballers' wages, on a team-by -basis. Analysing this, you would easily see where the highest disparity in wages are and in which teams.

In a similar way to visualising histograms in Matplotlib, the commands for boxplots are as follows, for both a dataframe and series, i.e. **series.plot.box()** and **dataframe.plot.box()**. To visualise the distribution within each column, as shown in Figure 1.24 below, we have created a boxplot of six trials with 20 observations of a unified random variable.

Example

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.rand(20, 6), columns=["U", "V", "W", "X", "Y", "Z"])

df.plot.box();
```

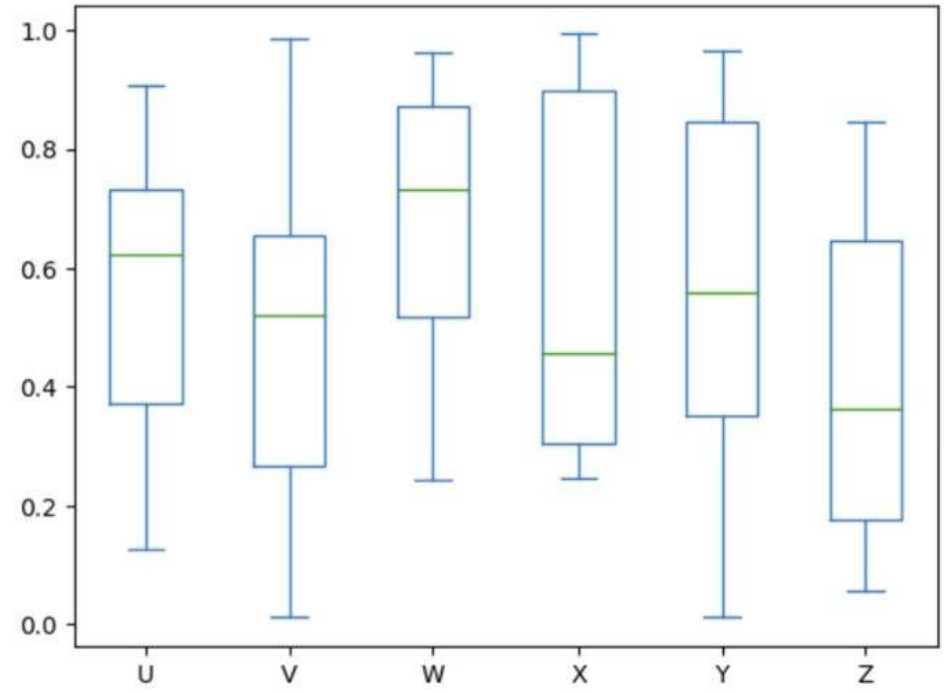


Figure 1.24. Basic box plot comparing multiple trials by Arden University. (Accessible code).

When the results are in the format shown in Figure 14 above, it can be difficult to determine the parameters that we want to show. Therefore, we can alter the colour of the parameters so that it is easy for us to determine what is what, as Figure 1.25 below demonstrates.

Example

```
color = {
    "boxes": "black",
    "whiskers": "blue",
    "medians": "red",
    "caps": "orange",
}


df.plot.box(color=color, sym="r+");
```

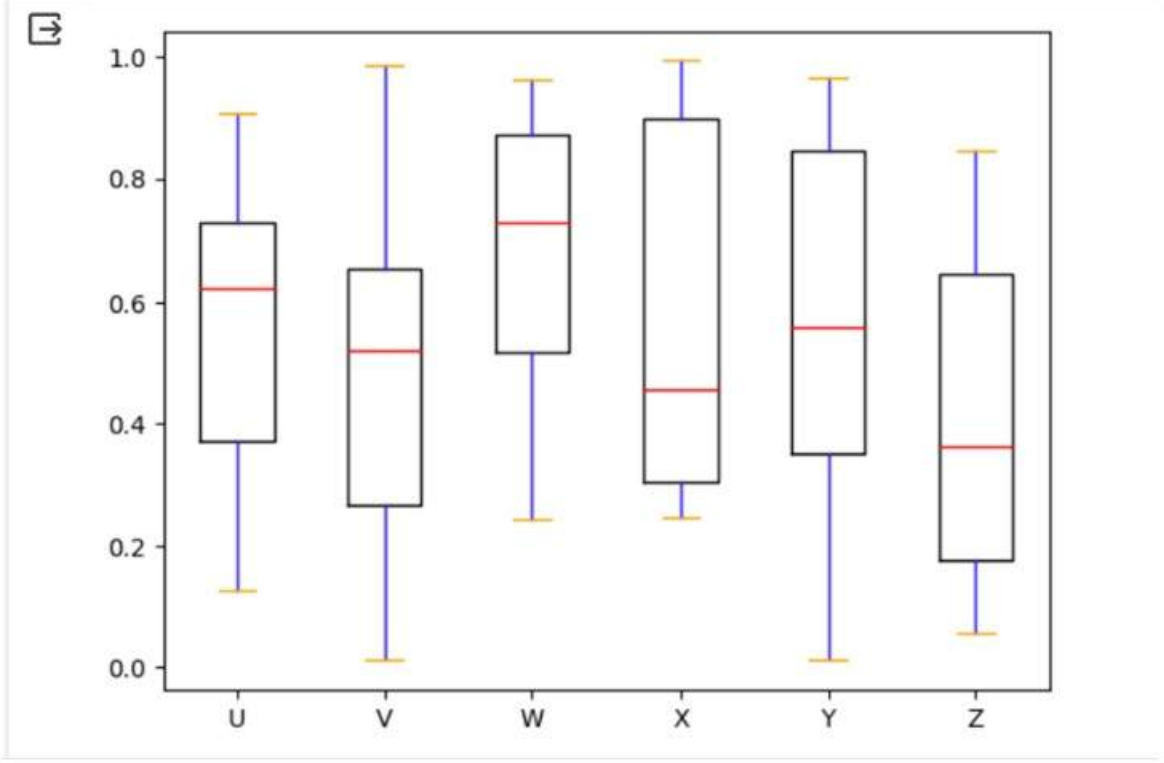


Figure 1.25. Colour differentiation to shown different parameters within a box plot, by Arden University. (Accessible code).

We can orientate the box plots differently too, to add a certain amount of effect to the visualisation, if this helps to portray our findings in a more easily readable manner. Figure 1.26 below demonstrates how we can orientate the box plot horizontally.

Example

```
df.plot.box(vert=False, positions=[2, 4, 6, 5, 7, 9]);
```

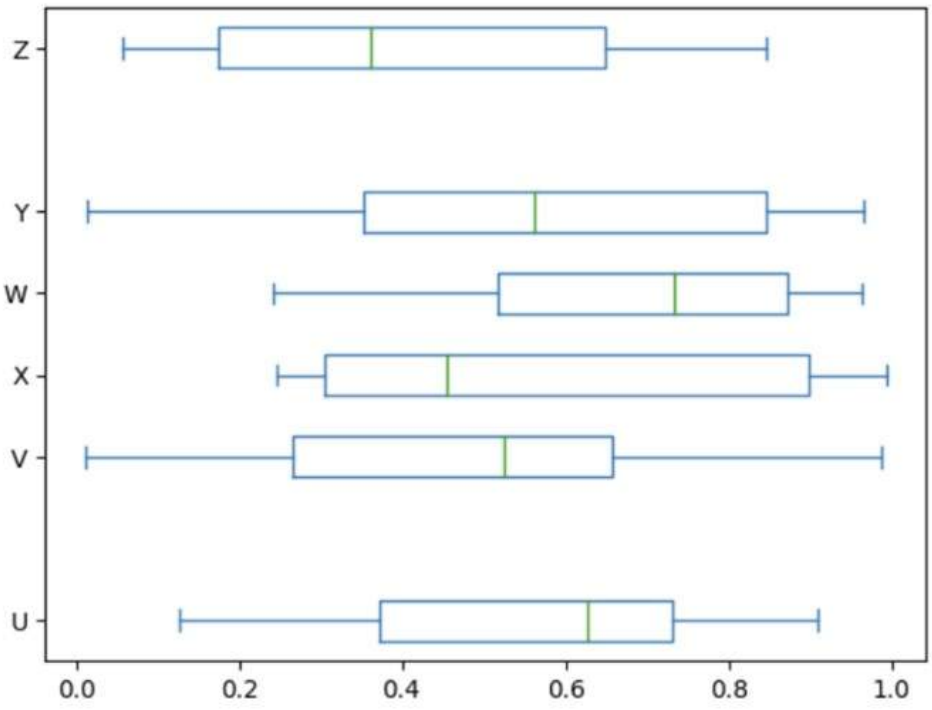


Figure 1.26. Alternative box plot orientation, by Arden University. (Accessible code).

Finally, we can stratify a box plot to create identified groupings and pass sub-sets of columns to the groups by multiple columns, for easier data recognition, as Figure 1.27 shows below.

Example

```
df = pd.DataFrame(np.random.rand(10, 3), columns=["Col3", "Col4", "Col5"])

df["V"] = pd.Series(["1", "1", "1", "1", "1", "2", "2", "2", "2", "2"])

df["W"] = pd.Series(["1", "2", "1", "2", "1", "2", "1", "2", "1", "2"])

bp = df.boxplot(column=["Col3", "Col4"], by=["V", "W"])
```

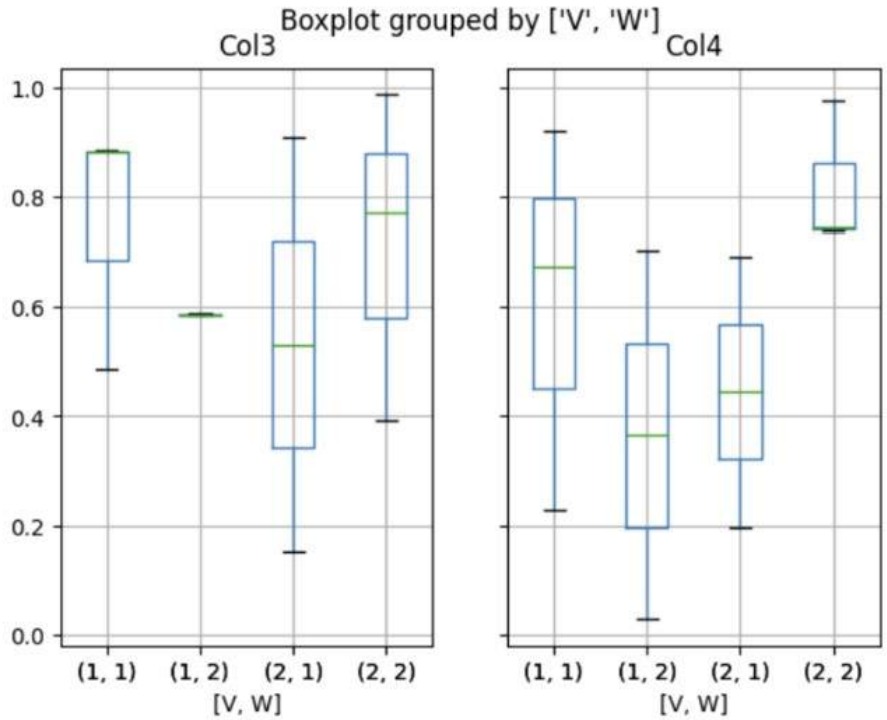
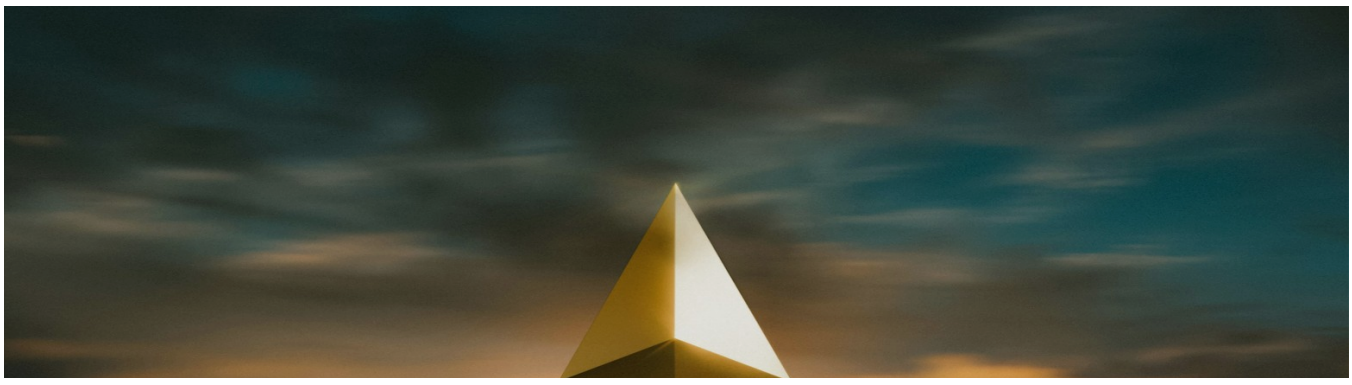


Figure 1.27. Visualisation of a stratified box plot, by Arden University. (Accessible code).

Continue



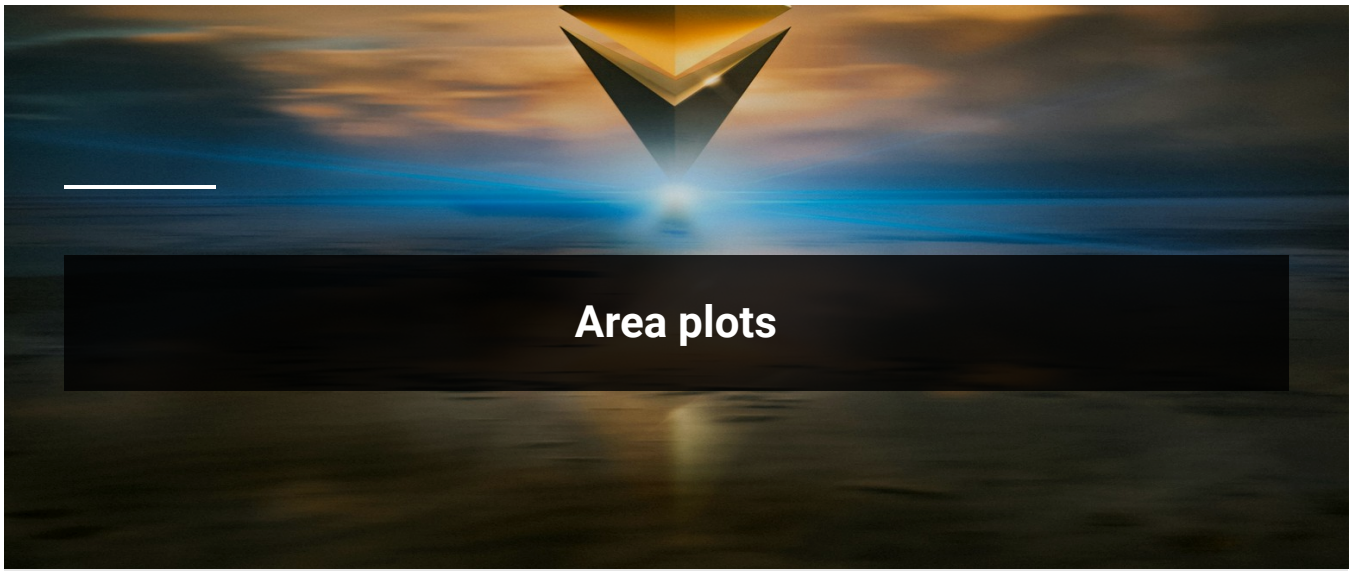


Image from Articulate Rise 360 Content Library.

An area chart is similar to a line graph, except the area underneath a line chart is clear, but with an area chart it is filled in with colour from the line to the axis. These charts are heavily used within sales data, where analysts are investigating the accumulative total sales over time. Within Matplotlib, by default, the area charts are stacked and all columns within the data must be either positive or negative to obtain results. When there is a data point within the data set such as NAN, the algorithm will be default and insert a 0 (zero). If we want to drop, or fill data within the data set, we must do this first before creating the area chart. We will now show an example of this particular chart so that we can visualise the theory above. In the example shown in Figure 1.28, we have three trials and 15 observations from a randomly created data set.

Example

```
df = pd.DataFrame(np.random.rand(15, 3), columns=["W", "X", "Y"])

df.plot.area();
```

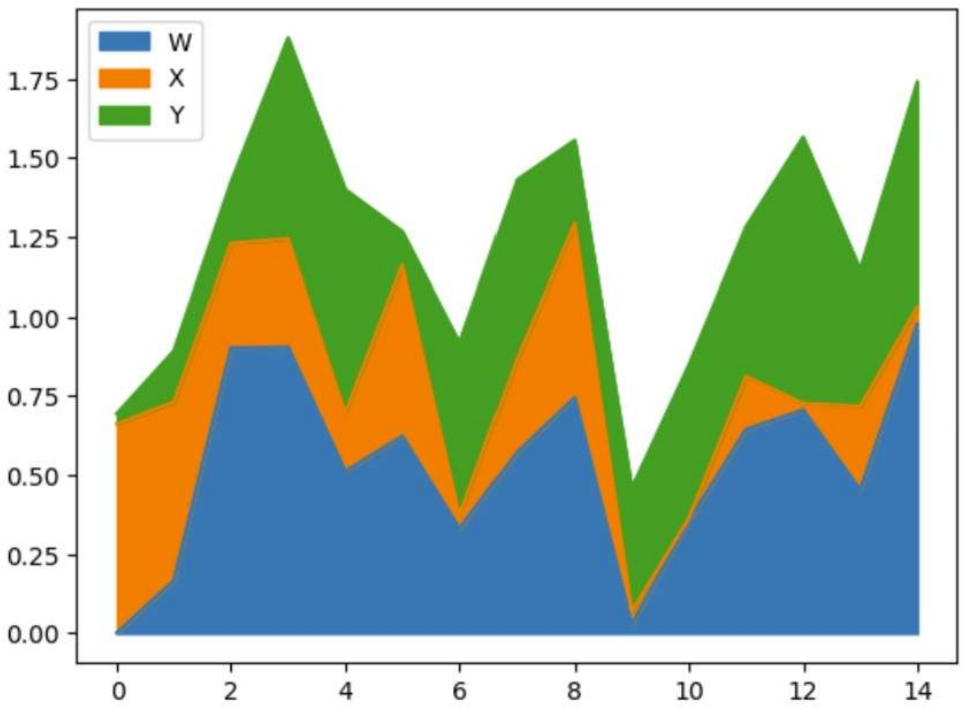
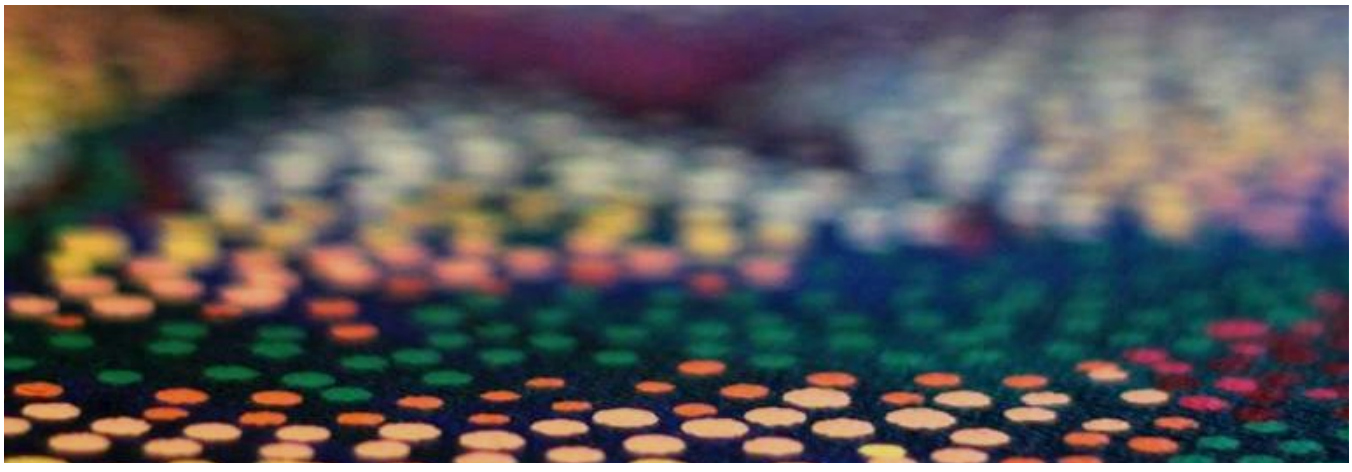


Figure 1.28. Example of an area plot from a randomly generated data set, by Arden University. (Accessible code).

Continue



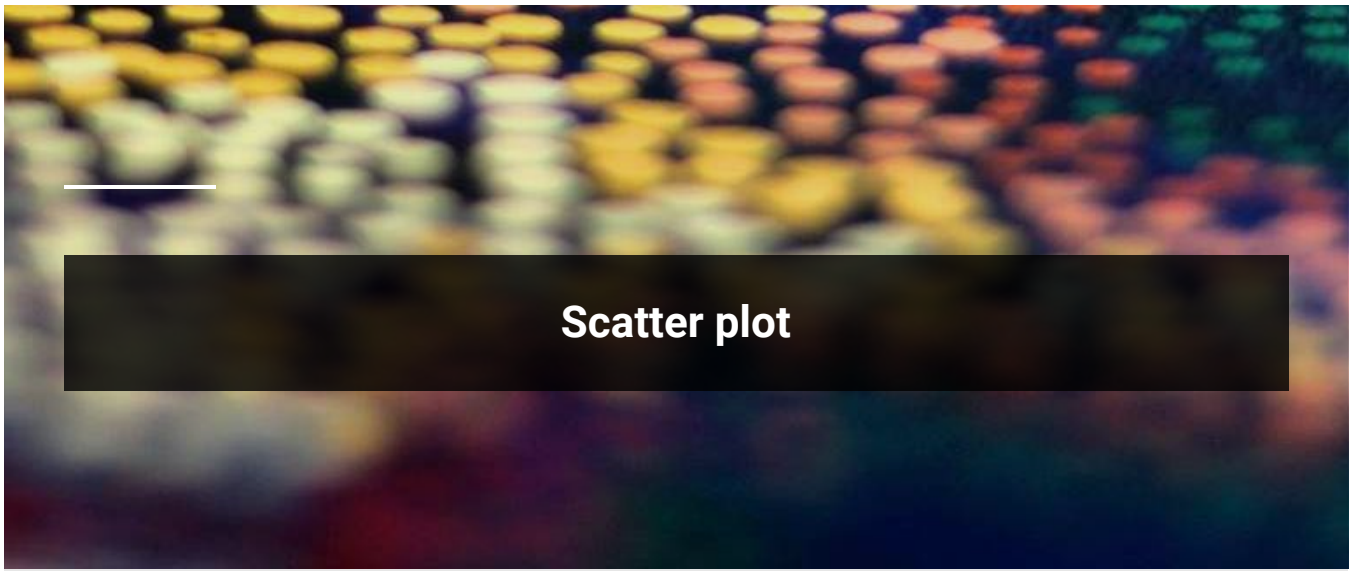


Image from Articulate Rise 360 Content Library.

Primarily, a scatter plot is used when we wish to show the relationship, if any, between numerical values. The dots represent numerical data points which can then be studied on either an individual basis, or as a collective. Scatter plots are ideal when studying correlations between variables: if the scatter plot is well spread out, this is a sign that there could be little or no correlation. If the scatter plot is clustered together, this is a sign that there could possibly be a correlation between two or multiple variables. Relationships between variables can be described as strong, weak, linear or non-linear. Not only are scatter plots useful for identifying correlations, they are also useful at highlighting any outliers within the data set.

Within Matplotlib, scatter plots are generated using the **dataframe.plot.scatter()** command. As stated earlier, scatter plots require numerical values from the X and Y variables. In Figure 1.29 below is an example of how to generate a scatter plot using the famous open-source Iris data set.

Example

```
df = pd.DataFrame(np.random.rand(40, 4), columns=["W", "X", "Y", "Z"])

df["species"] = pd.Categorical(
    ["setosa"] * 10 + ["versicolor"] * 10 + ["virginica"] * 20
)


df.plot.scatter(x="W", y="X");
```

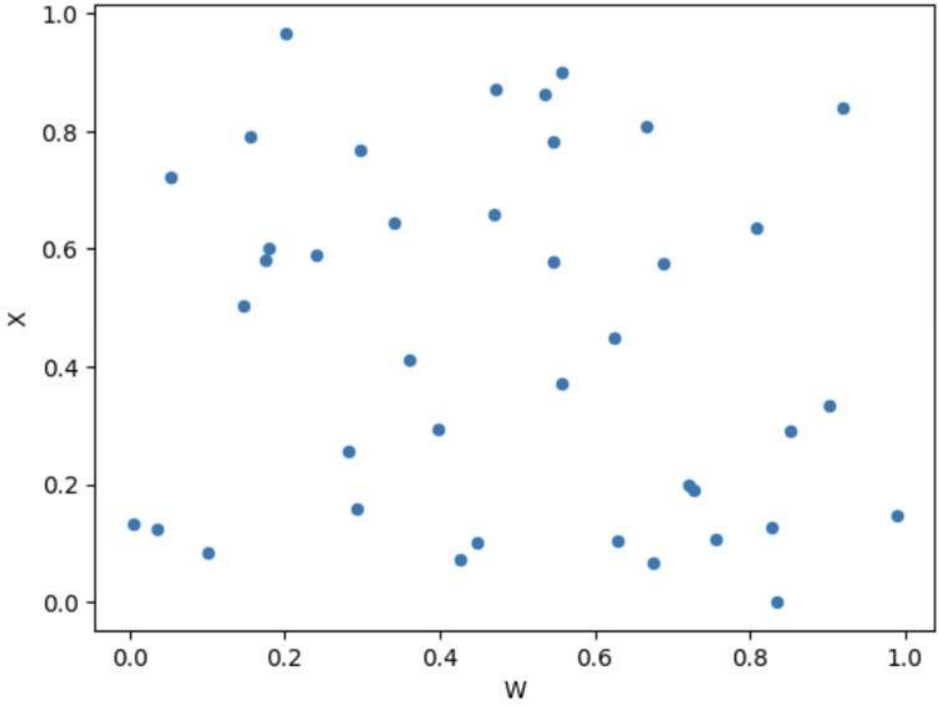


Figure 1.29. Basic scatter plot using the open-source Iris data set, by Arden University. (Accessible code).

In Figure 1.29 above, we have demonstrated the result of one scatter plot of two variables, but if we wanted to include another scatter plot, in addition to our original, how do we check for relationships between the two plots? This is no problem as we can combine the relationships between variables on two different scatter plots, as shown in Figure 1.30 below.

Example

```
ax = df.plot.scatter(x="W", y="X", color="Red", label="Group 5")

df.plot.scatter(x="Y", y="Z", color="Green", label="Group 6", ax=ax);
```

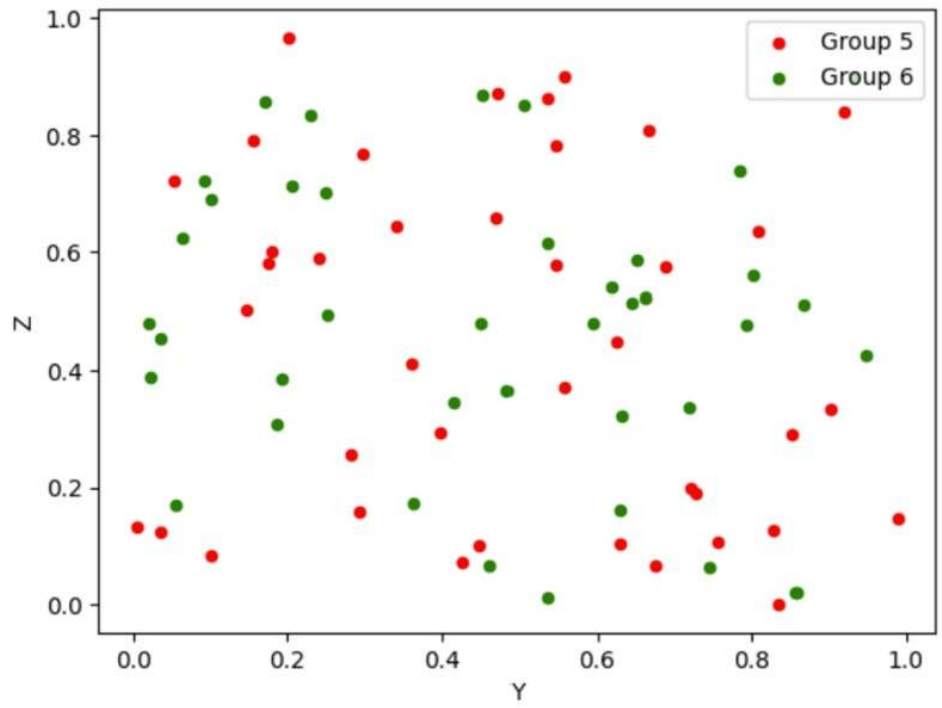


Figure 1.30. Imposing an additional scatter plot on top of an original, by Arden University. (Accessible code).

When creating visualisations, it is always advantageous to include as much information as you can without cluttering the graphic. In other words, only include information that helps the viewer to quickly understand what they are seeing. Sometimes you will find colour bars to the side of the scatter plot which helps the viewer to quickly identify certain elements within the visualisation. Figure 1.31 shown below demonstrates how useful the inclusion of a colour bar might be, using the Iris data set again.

Example

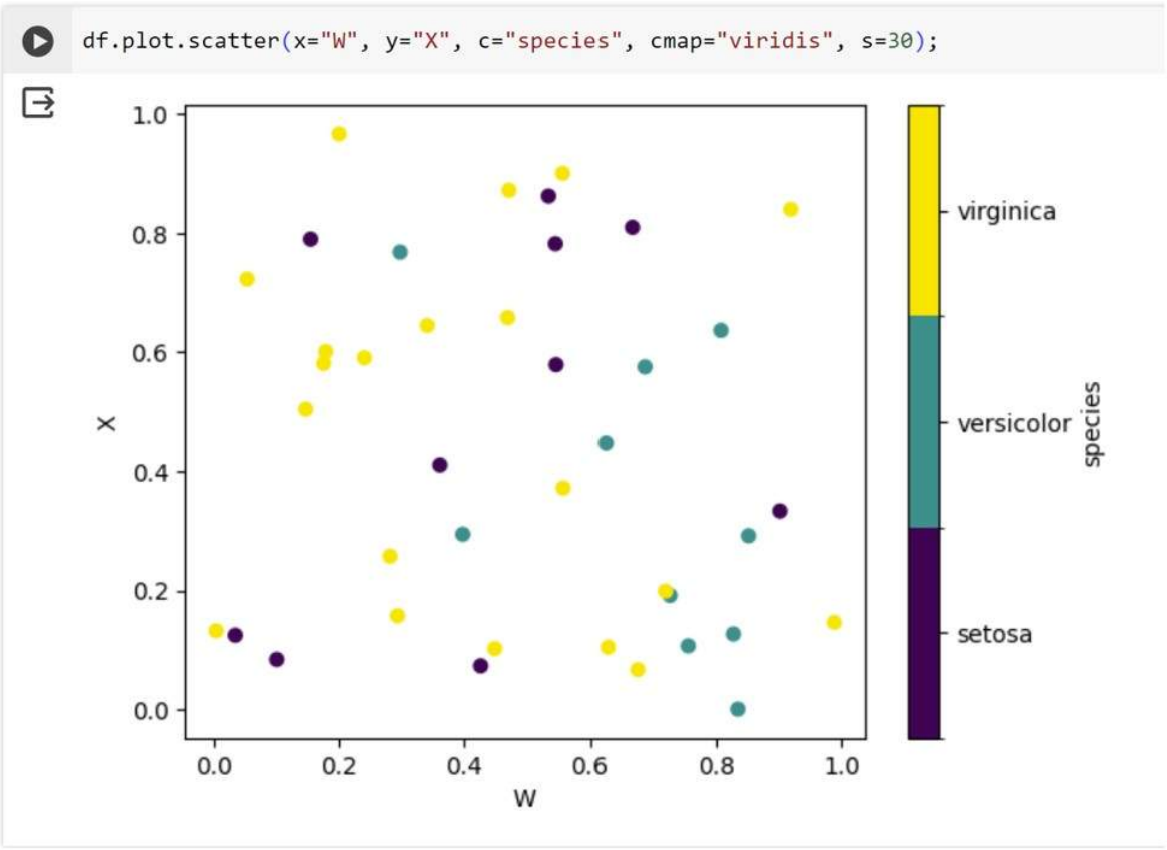


Figure 1.31. Scatter plot demonstrating multiple Iris data with associated colour bar for quick reference, by Arden University. (Accessible code).

Continue





Image from Articulate Rise 360 Content Library.

Pie charts will be the final basic plot visualisation that we shall demonstrate within this topic. There is a debate amongst the visualisation environment that pie charts should be avoided when trying to represent data. There are plenty of arguments for and against this view and you are encouraged to research this dilemma for yourselves to obtain an opinion of your own. But whether you agree with them or not, we shall be going through them here.

When pie charts are used in the correct context, they can be quite effective, in my opinion. However, care must be taken as to not include too many categories. If you do, the pie chart has a tendency to become cluttered and messy. Hopefully, by the end of this pie chart section you will have a firm understanding of what I mean. In Figure 1.32 below, we shall create a simple pie chart from randomly created data, consisting of four variables. Remembering that a pie chart is like a cake, therefore the entire data set will make up the entire cake, or 100%. The slices, both within the cake and the actual pie chart, represent a percentage of the cake.

Example

```
series = pd.Series(3 * np.random.rand(4), index=["W", "X", "Y", "Z"], name="Pie Chart")

series.plot.pie(figsize=(4, 4));
```

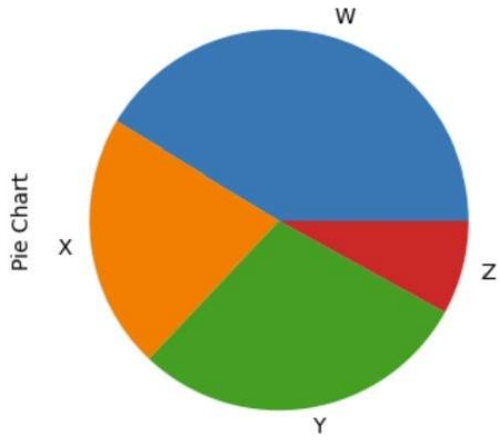


Figure 1.32. Basic pie chart from randomly created data, by Arden University. (Accessible code).

Example

```
df = pd.DataFrame(
    3 * np.random.rand(4, 2), index=["W", "X", "Y", "Z"], columns=["1", "2"]
)

df.plot.pie(subplots=True, figsize=(8, 4));
```

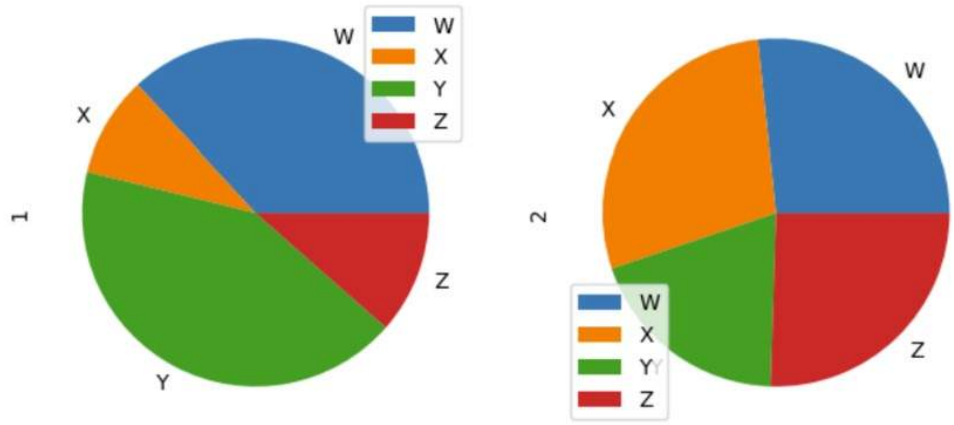


Figure 1.33. Comparative pie charts upon the same categories, by Arden University. (Accessible code).

Example

```
series.plot.pie(
    labels=["ww", "xx", "yy", "zz"],
    colors=["r", "m", "y", "c"],
    autopct="%.2f",
    fontsize=15,
    figsize=(6, 6),
);
```

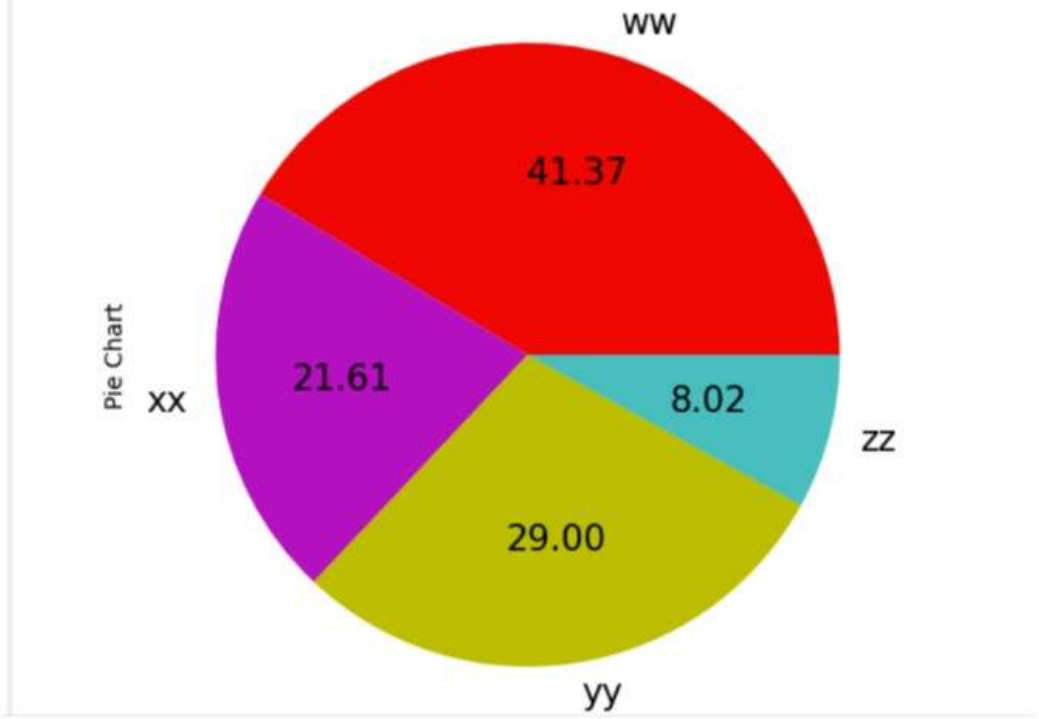


Figure 1.34. Pie chart with inserted percentage values, by Arden University. (Accessible code).

Hurrah!

This concludes the topic on basic chart plotting within Matplotlib. We have taken a steady but in-depth look at bar charts, histograms, line charts, area charts, box and whisker plots and pie charts. It is important to remember that great and clear data visualisations can really help to improve the results that you give to stakeholders and can enhance your data investigation.

Continue

Topic 3 - Activity 1

Reading



Chapter 3: Human Perception and Information Processing (pp. 81-138) in *Interactive Data Visualisation* by Ward, Grinstien and Keim

Chapter 3: Snakes, Bears & Other Numerical Beasts: NumPy, SciPy & pandas (pp. 137-178) in *Statistics and Data Visualisation with Python* by Rogel-Salazar

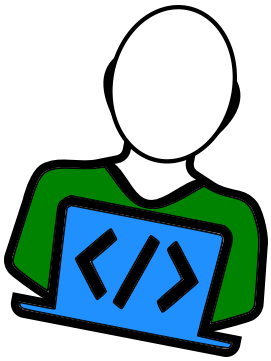
Wider reading: Chapter 3: The Graphics (pp. 54-81) in *Data Visualisation made Simple* by Sosulski

Continue

Topic 3 - Activity 2

Using data = { 'C': 80, 'C++': 30, 'Java': 76, 'Python': 49, 'Fortran90': 14} create a conventional bar chart, horizontal bar chart and pie chart in Matplotlib's default colours, or choose your own.

Next, using hierarchical ideology, rearrange the result and use an impactful colour palette to emphasise your results. When you have done this, post your visualisations into the group's Discussion Forum for further discussions with your peers and tutor.



Continue

Important point

Deciding which basic plot to choose can be difficult. However, now that we have discussed this at length, you are much better positioned in deciding which plots to choose, dependent on the data set and investigation.

Continue

TOPIC 3 SUMMARY

Well done, you have completed Topic 3.

Here, we have focused on re-capping basic chart plotting within Matplotlib. We have again taken a steady look at bar charts, histograms, line charts, area charts, box and whisker plots and pie charts.

Moving on, we can now give multiple examples of a further, more advanced visualisation library within Python, namely Seaborn.

Continue to Topic 4

Lesson 6 of 7

Topic 4: Introduction to Seaborn



Image by mwaskom from Github.com.

As we have already demonstrated in Topic 3, basic plot types are relatively easy to produce using Matplotlib. In Topic 4 we will give an introduction to Seaborn within a Python environment.

Initially, Seaborn is a data visualisation library based on Matplotlib. The main difference between Seaborn and Matplotlib is that Seaborn offers a higher-level interface for visualising statistical graphs. The library gives very good default styles and a higher-intensity colour map to make data scientists' plots more impressive, which we shall now demonstrate.

Continue

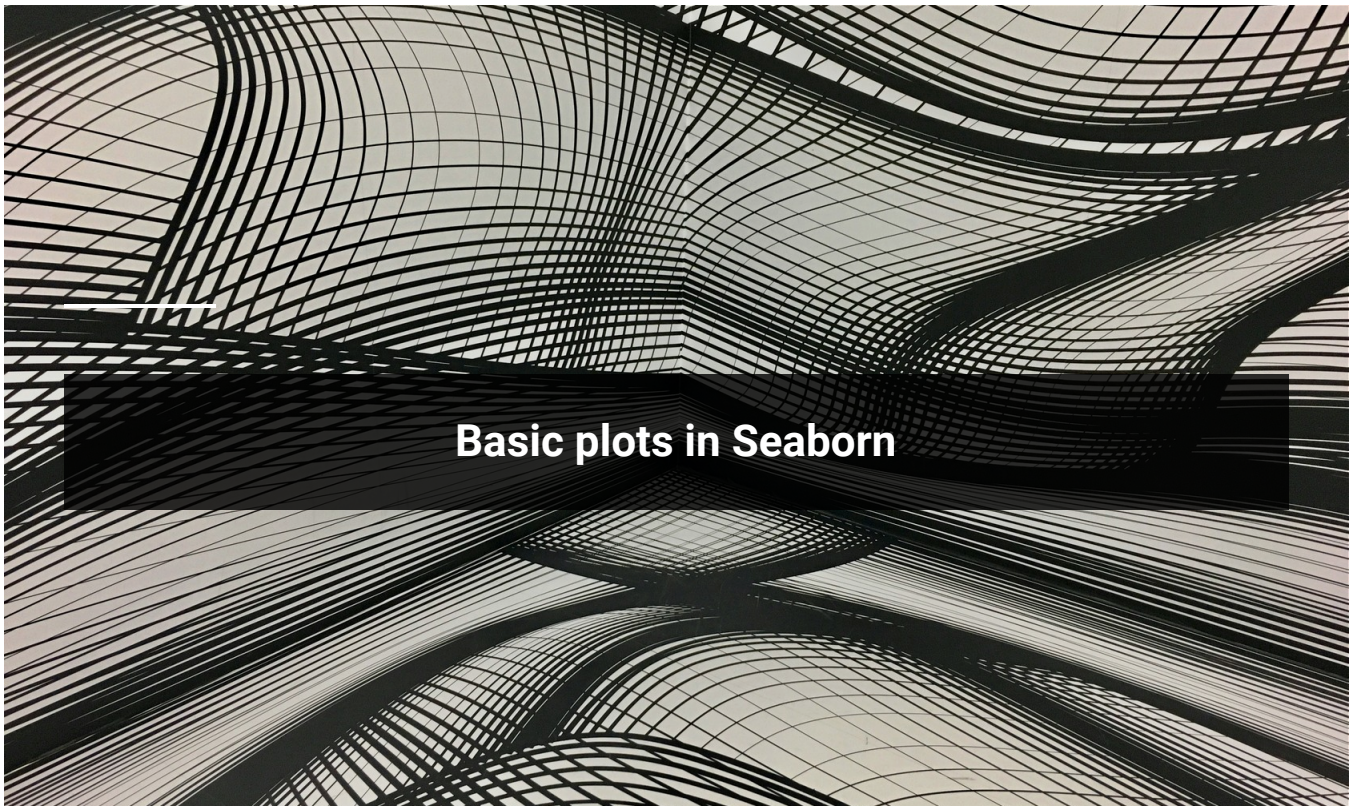


Image from Articulate Rise 360 Content Library.

Histplot

Histplot is used to plot a histogram of univariate data, in other words a single variable. As can be seen in the example below in Figure 1.35, **histplot** takes several functions to produce the visualisation, but the important ones to note are stated below.

- Data: this the array, series or dataframe we wish to visualise, therefore an important parameter to define.
- X: defines the column in the data to use for the histogram. You can specify this by column name if working within a data frame.
- Y: similar to X, defines the column when a bivariate histogram is to be created. As a default, this function is set to none, which in turn means that a univariate histogram will be displayed.
- Bins: are specified when dividing the data up into intervals ready for plotting. By default, this is set to auto, which in turn employs an algorithm to determine the optimal number of bins required.
- Kde: represents the kernal density estimate , in other words the line of best fit onto the histogram, which is used for distribution purposes.

Example

```python
import numpy as np
import seaborn as sns

sns.set(style="white")

# Generating a random univariate dataset
rs = np.random.RandomState(20)
d = rs.normal(size=50)

# Plot a simple histogram with kde
sns.histplot(d, kde=True, color="c")
```

```
<Axes: ylabel='Count'>
```

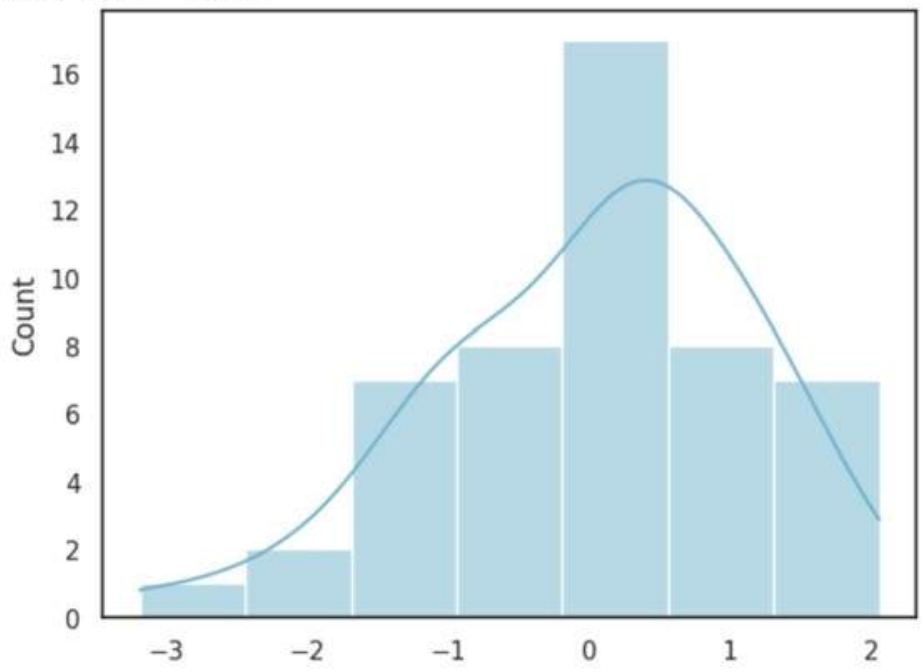


Figure 1.35. Histogram using the histplot function in Seaborn, by Arden University. (Accessible code).

Continue

Displot

Displot can be applied to visualise univariate distributions, see Figure 1.36. As with histplot, the function takes several parameters which are similar, such as kde and bins. There are additional parameters, however, and these are detailed below.

- A: this is the parameter similar to data in the histplot. In other words, it is the array, dataframe or series.
- Hist: this parameter informs the algorithm as to whether a histogram is requested or not. By default, this parameter is set to true.

Example

```python
import numpy as np
import seaborn as sns

sns.set(style="white")

# A random univariate dataset
rs = np.random.RandomState(10)
d = rs.normal(size=50)

# Define the colours to use
colors = ["g", "b", "r"]

# A histogram with multiple colours
sns.distplot(d, kde=True, hist=True, bins=5,
             rug=True,hist_kws={"alpha": 0.5,
                                "color": colors[0]},
             kde_kws={"color": colors[1], "lw": 2},
             rug_kws={"color": colors[2]})
```

```
  sns.distplot(d, kde=True, hist=True, bins=5,
<Axes: ylabel='Density'>
```

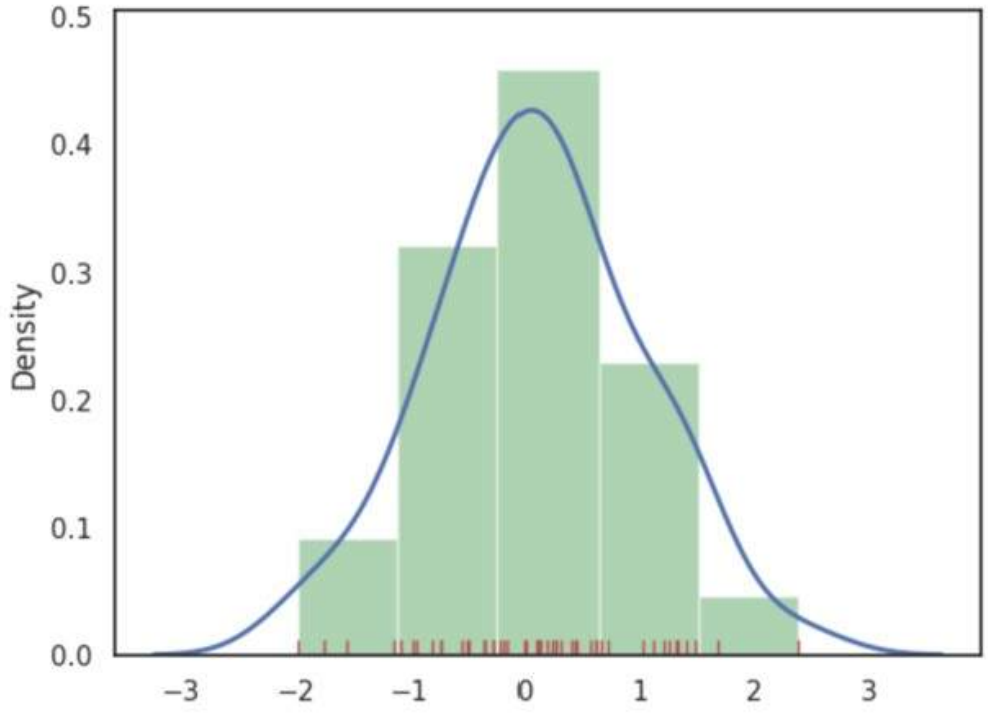


Figure 1.36. Histogram using the **displot()** function in Seaborn, by Arden University. (Accessible code).

Continue

Lineplot

Lineplot is arguably the most basic of plots within the Seaborn library and used primarily to visualise time series data, as Figure 1.37 demonstrates.

Example

```python
import seaborn as sns

sns.set(style="dark")
fmri = sns.load_dataset("fmri")

# Plotting responses for different events and regions
sns.lineplot(x="timepoint",
             y="signal",
             hue="region",
             style="event",
             data=fmri)
```

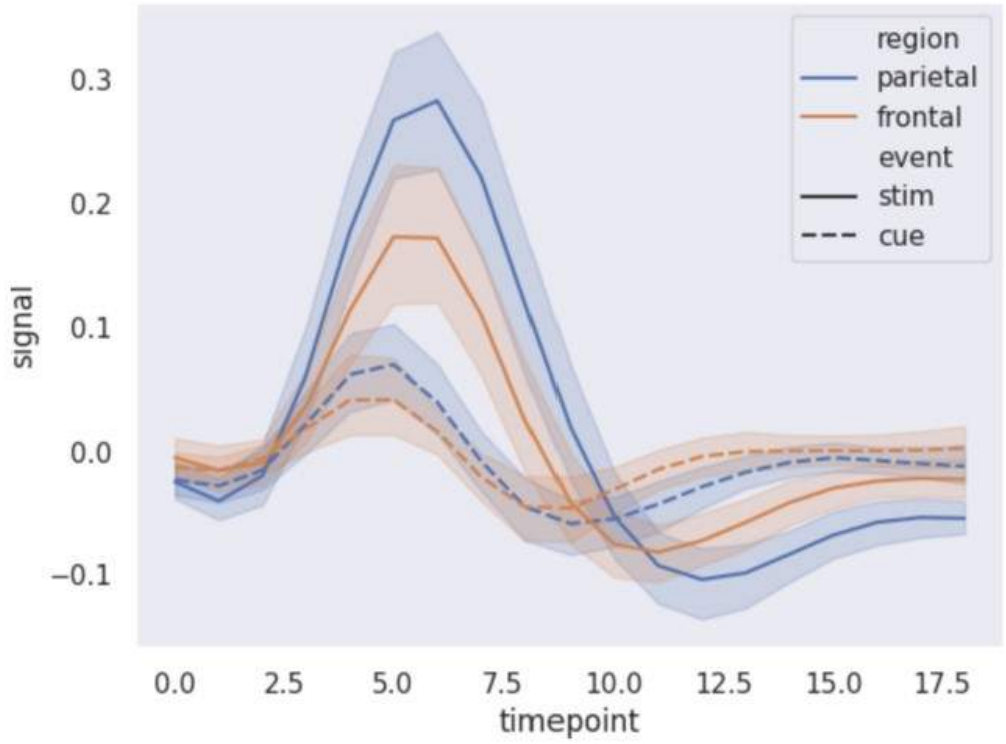


Figure 1.37. Multiple line plots using **lineplot()** in Seaborn, by Arden University. (Accessible code).

Continue

Lmplot

In addition to the lineplot function, there is another basic plot within the Seaborn library, namely Lmplot. Lmplot gives a line for best fit within a 2D linear regression study, where the x and y variables are set along the horizontal and vertical axis, respectively. An example of which can be seen below in Figure 1.38 using the combined Anscombe quartet data set.

Example

```
import seaborn as sns

sns.set(style="ticks")

# Loading dataset
df = sns.load_dataset("anscombe")

# Resulting linear regression model
sns.lmplot(x="x", y="y", data=df)
```

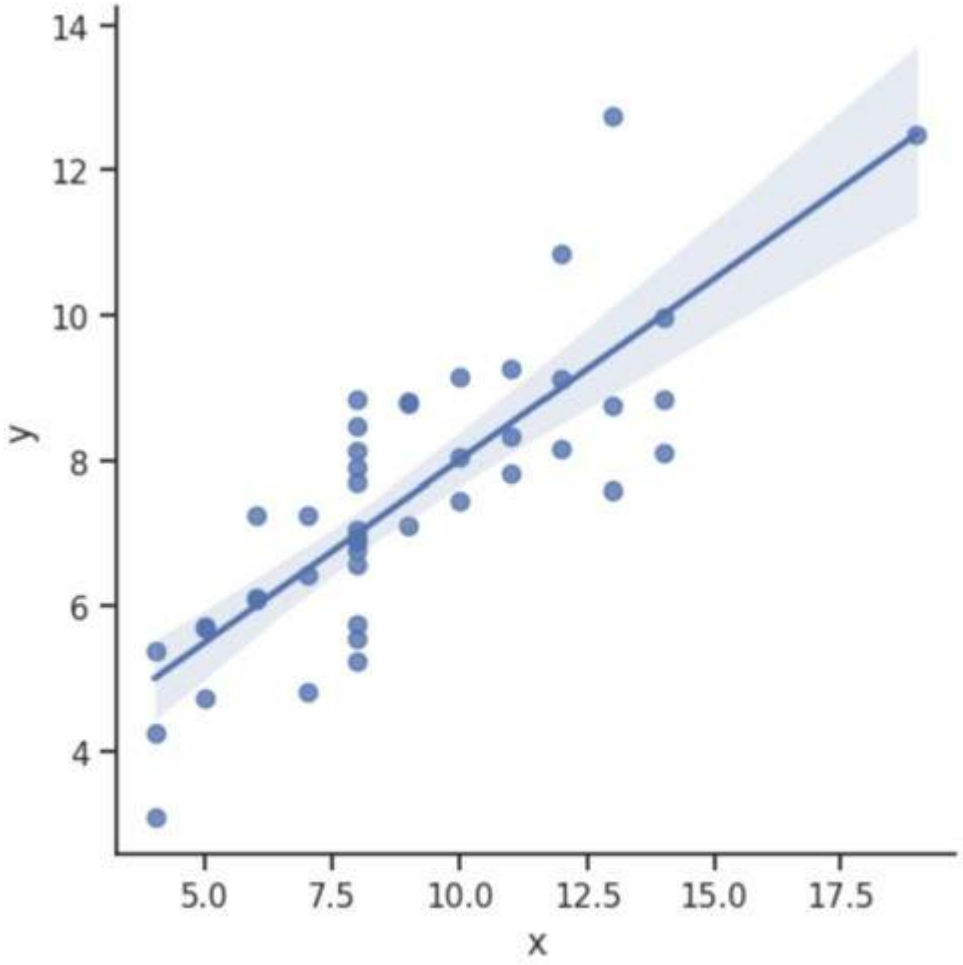


Figure 1.38. Using the **Lmplot()** function in Seaborn, by Arden University. (Accessible code).

Continue

Stripplot

The stripplot function is a variant of the scatter plots which is available to the data scientist in Seaborn. As we are aware from a previous lesson, categorical data is plotted along the x axis and the corresponding values shown on the y axis. The **stipplot()** function informs the coding environment which scatter plot is being requested and the **.set()** function uses the *x* and *y* axis labels. Finally, the **.show()** function, which we have seen many times before, allows the analyst to visualise the plot. In the example below in Figure 1.39, we have utilised the in-built Iris data set.

Example

```
import matplotlib.pyplot as plt
import seaborn as sns

# Setting style of background
sns.set(style="whitegrid")

iris = sns.load_dataset('iris')

# Determining the attributes of the dataset which plot is to be made
ax = sns.stripplot(x='species', y='sepal_length', data=iris)

# giving title to the plot
plt.title('Iris')

# function to show plot
plt.show()
```

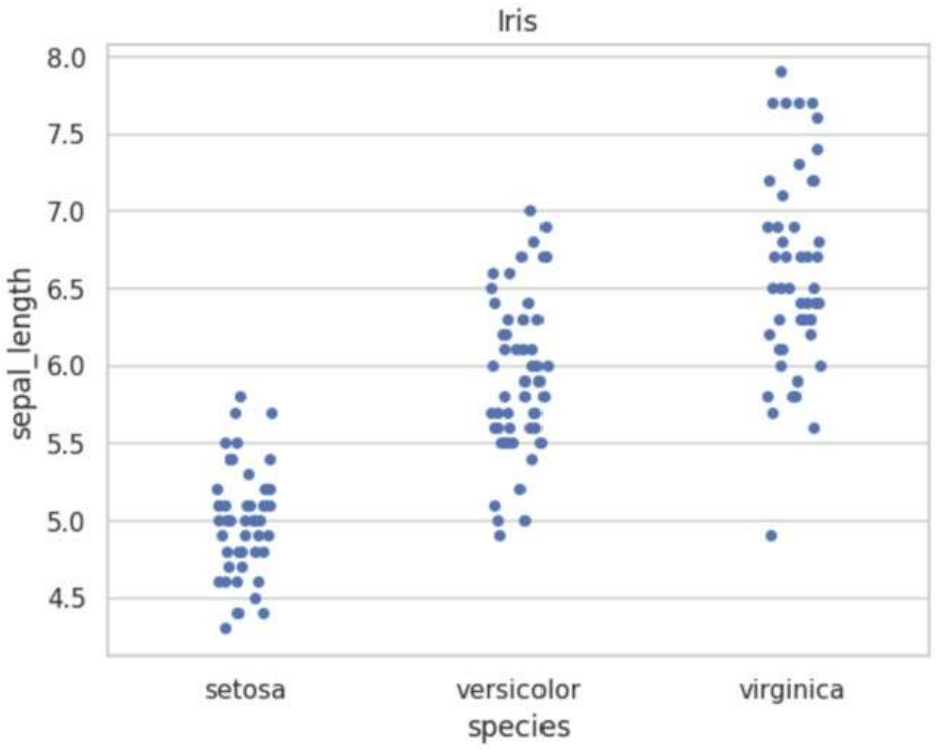


Figure 1.39. Demonstrating **stripplot()** in Seaborn, by Arden University. (Accessible code).

Continue

Swarmplot

As we shall see, the **swarmplot()** is very similar to **stripplot()**. The difference between the two plots, however, is that **swarmplot()** does not allow overlapping data points. In not allowing this it makes the plot a little easier to read, plus it removes the possible loss of information overlapping could generate. We shall continue using the Iris data set in the following two examples shown in Figures 1.40 and 1.41. However, notice in the latter Figure 1.41, we have alternated the dependent and independent variables.

Example

```
import matplotlib.pyplot as plt
import seaborn as sns

# use to set style of background of plot
sns.set(style="darkgrid")
iris = sns.load_dataset('iris')

# choosing the swarmplot
ax = sns.swarmplot(x='species', y='sepal_length', data=iris)

plt.title('Iris')
plt.show()
```

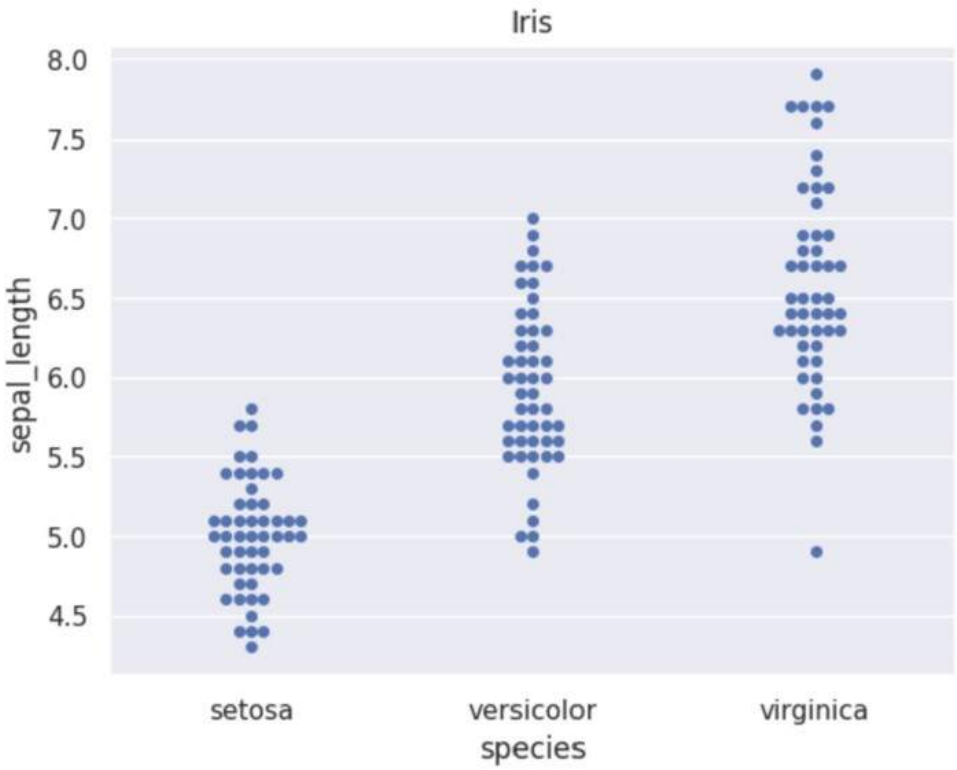


Figure 1.40. Using **swarmplot()** as an alternative to **stripplot()** for none overlapping data points, by Arden University. (Accessible code).

Example

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="darkgrid")
iris = sns.load_dataset('iris')

# Alternating the attributes
ax = sns.swarmplot(x='sepal_length', y='species', data=iris)

plt.title('Iris')

plt.show()
```

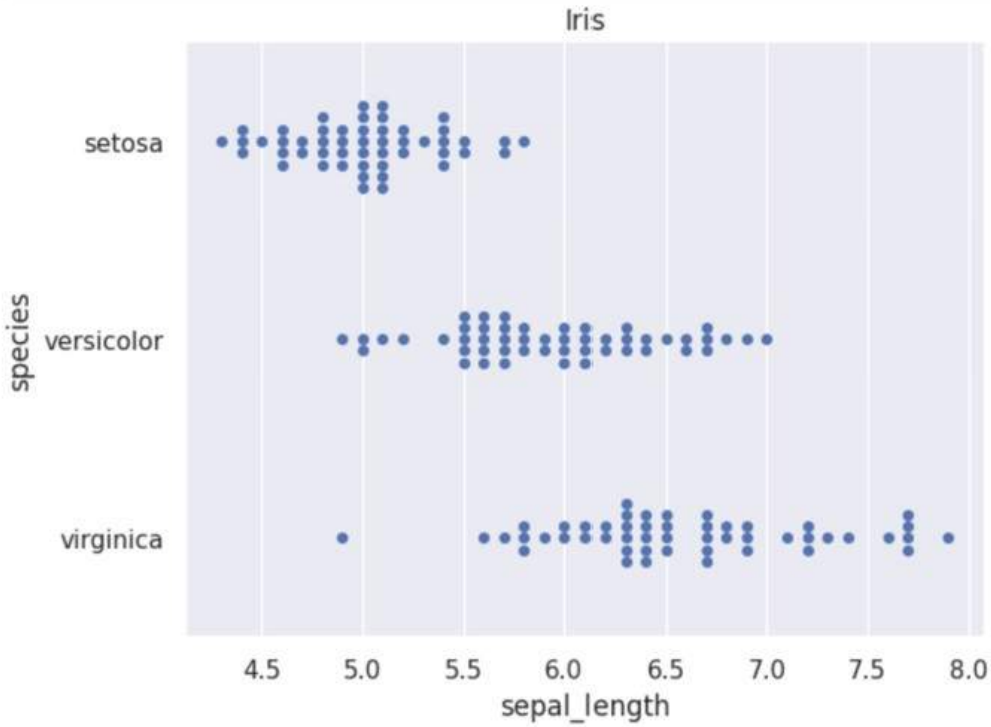


Figure 1.41. Alternating the dependent and independent variables in **swarmplot()**, by Arden University. (Accessible code).

Continue

Barplot

By default, the **barplot()** function uses the mean value of the categorical data as the aggregate. So that the analysist can visualise the data, a categorical column for *x* is chosen, as well as a numerical value for *y*. This in turn results in the function taking the mean per categorical column and then plots the results, as Figure 1.42 below shows on a fictitious restaurant data of male and female tips from the total bill.

Example

```
import seaborn as sns

df = sns.load_dataset('tips')

sns.barplot(x ='sex', y ='total_bill', data = df,
            palette ='tab10')
```

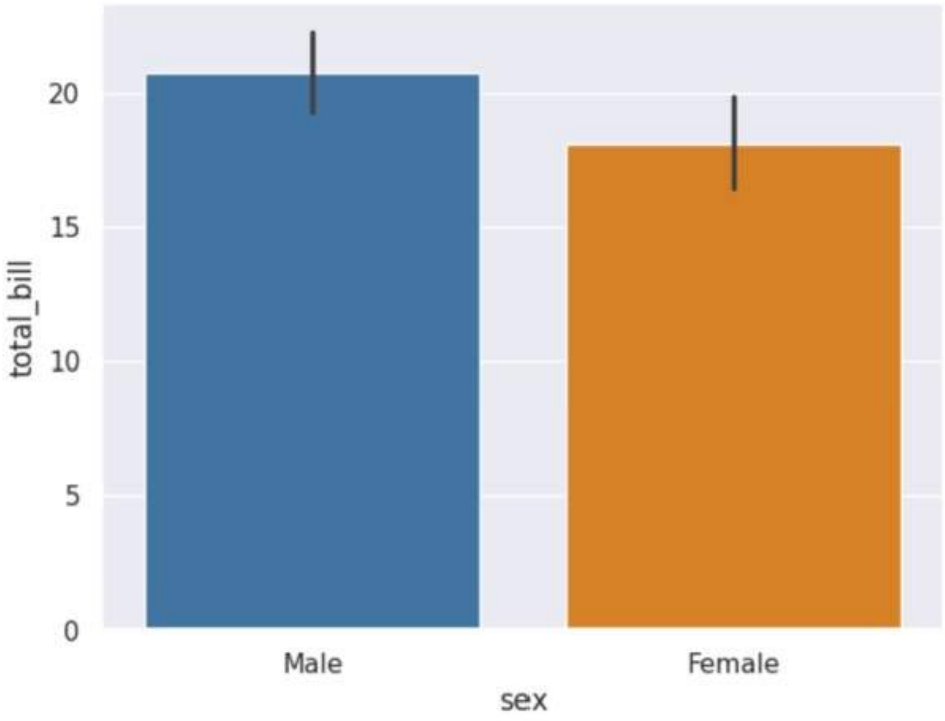


Figure 1.42. A resulting box plot from categorical data using the tab10 palette, by Arden University. (Accessible code).

Continue

Violinplot

The **violinplot()** is similar to the box plot we demonstrated in a previous lesson. The **violinplot()** function does, however, give the analyst a much more advanced visualisation and uses the kde to give a better representation of the data density distribution. In Figure 1.43 below, hue is used, along with the function split = true which allows for easier comparatives, as seen in Figure 1.42 above.

Example

```
# import the seaborn library
import seaborn as sns

# reading the dataset
df = sns.load_dataset('tips')
sns.violinplot(x='day', y='total_bill', data=df,
               hue='sex', split=True)
```

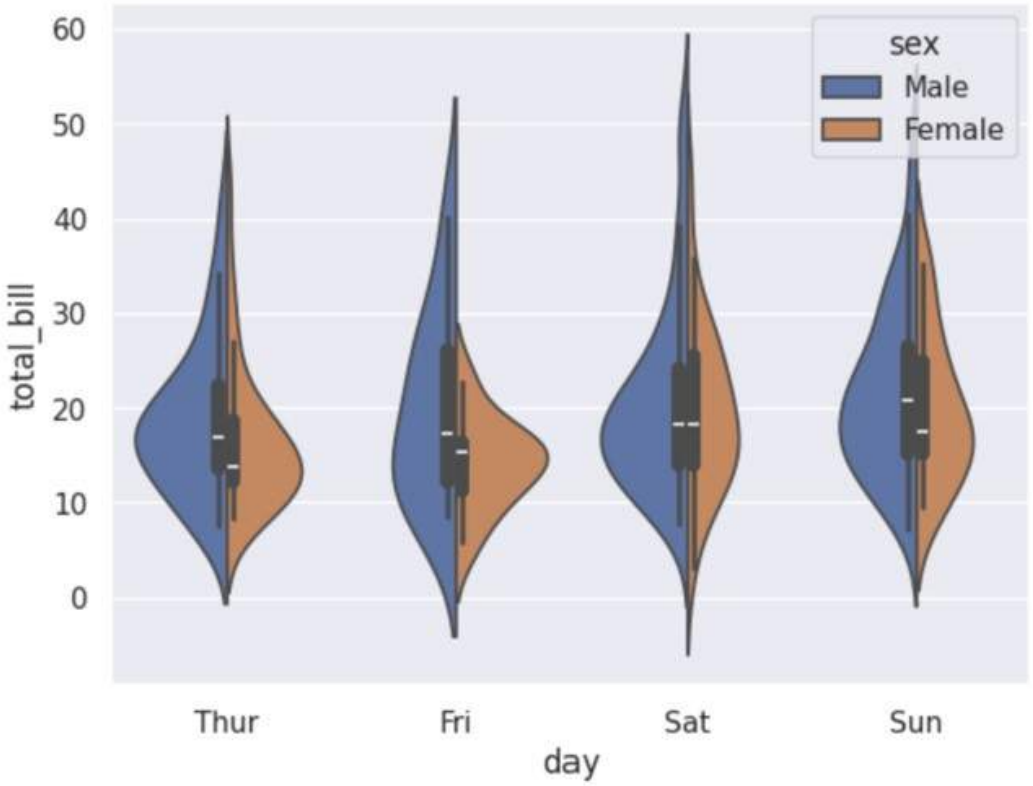


Figure 1.43. Using the **violinplot()** for further enhance categorical visualisations, by Arden University. (Accessible code).

Hopefully, it can be determined that the Seaborn library offers the data scientist higher-grade visual representation of the data being studied. We shall re-visit the Seaborn library in much greater detail as the module progresses.

Continue

Topic 4 - Activity 1

Seaborn library



Create a data visualisation of your choice using the Seaborn library. Now use the extensive Python colour palette, of which the Excel spreadsheet can be found below, to alter the colours of your visualisation. Dependent on what data set you used to produce the visualisation, what colours best represent the data study and the impact you are trying to create?

Post your visualisations, with differing colours, into the group's Discussion Forum so that your choices can be further discussed with your peers and tutor.

FILE

Python_Colour_Palette.csv

22.3 KB

Continue

Topic 4 - Activity 2

Reading



Chapter 4: Visualization Foundations (pp. 139-182) in *Interactive Data Visualisation* by Ward, Grinstien and Keim

Chapter 4: The Measure of All Things – Statistics (pp. 179-216) in *Statistics and Data Visualisation with Python* by Rogel-Salazar

Wider reading: Chapter 4: The Data (pp. 82-107) in *Data Visualisation made Simple* by Sosulski

Continue

TOPIC 4 SUMMARY

Well done, you have completed Topic 4.

In Topic 4, we have focused on introducing basic chart plotting in Seaborn. We have again taken a steady look at bar charts, histograms, line charts to name but a few. Hopefully, you can now determine that Seaborn offers the analyst advanced visualisation functions in Python. We shall re-visit Seaborn in greater depth as the module develops.

All topics are now concluded within this lesson.

Continue to 'Reading list'

Reading List

All reading and resources in this module are provided by the Arden University Library service. The full reading list for this module can be accessed at the link below. More books, journals and databases can be found on the Library Portal. If you need these resources in a different format, our accessibility tools can help.

Full reading list

You can find all the key, essential and wider reading and resources for this module on the dedicated reading list page on the Learning Success Hub.

VIEW READING LIST

References

Cited sources that do not appear in your reading list:

Yalcinkaya, M. and Singh, V., (2019). VisualCOBie for facilities management: A BIM integrated, visual search and information management platform for COBie extension. *Facilities* [online], 37 (7/8), pp. 502-524